

Lab Manual

Data Mining



By

Abdullah Azhar

2023-BS-AI-150

Submitted to

Sir Arham

Bachelor of Science in Artificial Intelligence
DEPARTMENT OF COMPUTER SCIENCES

Table of Contents

Lab 1: Knime Introduction	4
Real World Problem	4
Dataset.....	4
Practical Objectives:	4
Procedure:	5
Step 1. Create a New Workflow.....	5
KPI Summary:	10
Lab 2: CRISP-DM Methodology	11
Real World Problem	11
Dataset.....	11
Practical Objectives	11
Procedure.....	11
KPI Summary.....	14
Lab 3: Data Cleaning.....	15
Real World Problem	15
Dataset.....	15
Practical Objectives	15
Procedure.....	15
KPI Summary.....	18
Lab 4: Data Preprocessing	19
Real World Problem	20
Dataset.....	20
Practical Objectives	20
Procedure.....	20
KPI Summary.....	23
Lab 5: Market Basket Analysis.....	25
Real World Problem	25
Dataset.....	25
Practical Objectives	25
Procedure.....	25
KPI Summary.....	27
Lab 6: Apriori Algorithm.....	28

Real World Problem	29
Dataset.....	29
Practical Objectives	29
Procedure.....	29
KPI Summary.....	31
Lab 7: FP-Growth Algorithm	33
Real World Problem	33
Dataset.....	33
Practical Objectives	33
Procedure.....	33
KPI Summary.....	36
Lab 8: Python Basics for Data Mining.....	37
Real World Problem	37
Dataset.....	37
Practical Objectives	37
Procedure.....	37
KPI Summary.....	40

Lab 1: Knime Introduction

Real World Problem

A retail store has accumulated sales data across multiple product categories and needs a structured way to analyse its performance. The management requires a clear understanding of overall revenue, the average value of each order placed, which categories drive the most income, and which individual products are the highest earners. This lab uses KNIME to address these needs through a series of visual, node-based operations on a CSV dataset..

Dataset

The dataset is a Retail Sales CSV file containing one row per transaction. I

The screenshot shows the Kaggle dataset page for 'Retail Insights: A Comprehensive Sales Dataset'. The page includes a sidebar with navigation options like Home, Competitions, Datasets, Models, Benchmarks, Game Arena, Code, Discussions, Learn, and More. The main content area displays the dataset details, including a 'data.csv' file (1.25 MB) and a 'Data Explorer' section. The 'Data Explorer' shows a summary of the dataset, including the number of files (2) and columns (48). A table of sample data is also visible, showing columns like Order No, Order Date, Customer Name, Address, City, and State.

Order No	Order Date	Customer Name	Address	City	State
Unique identifier for each order.	Date when the order was placed.	Name of the customer placing the order.	Customer's address.	City where the customer is located.	State
881 unique values		Christina Vanderza... 1%	180 High Street,Wi... 5%	Sydney 72%	NSW
		Michael Oakman 1%	188 Pitt Street,Syd... 4%	Melbourne 28%	VIC

Practical Objectives:

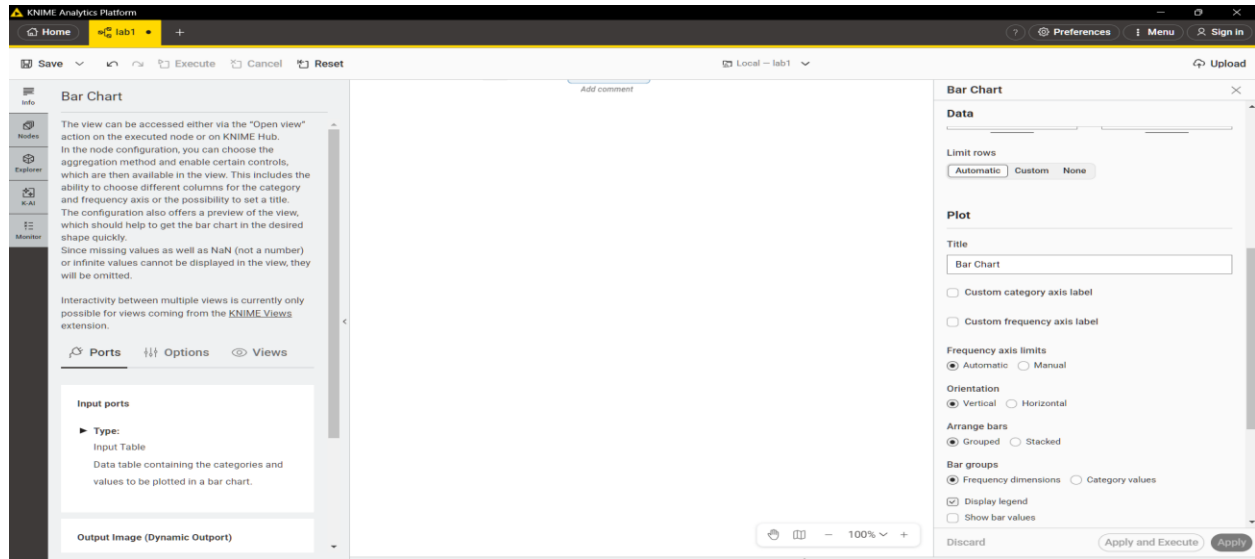
By the end of this lab, students will have hands-on experience with the core features of KNIME Analytics Platform. Specifically, students will be able to:

1. Import a CSV dataset into a KNIME workflow using the CSV Reader node.
2. Inspect and correct column data types where necessary.
3. Generate descriptive summary statistics to understand data distribution.
4. Create a derived Revenue column using the Math Formula node.
5. Compute four business KPIs using GroupBy and Sorter nodes.

Procedure:

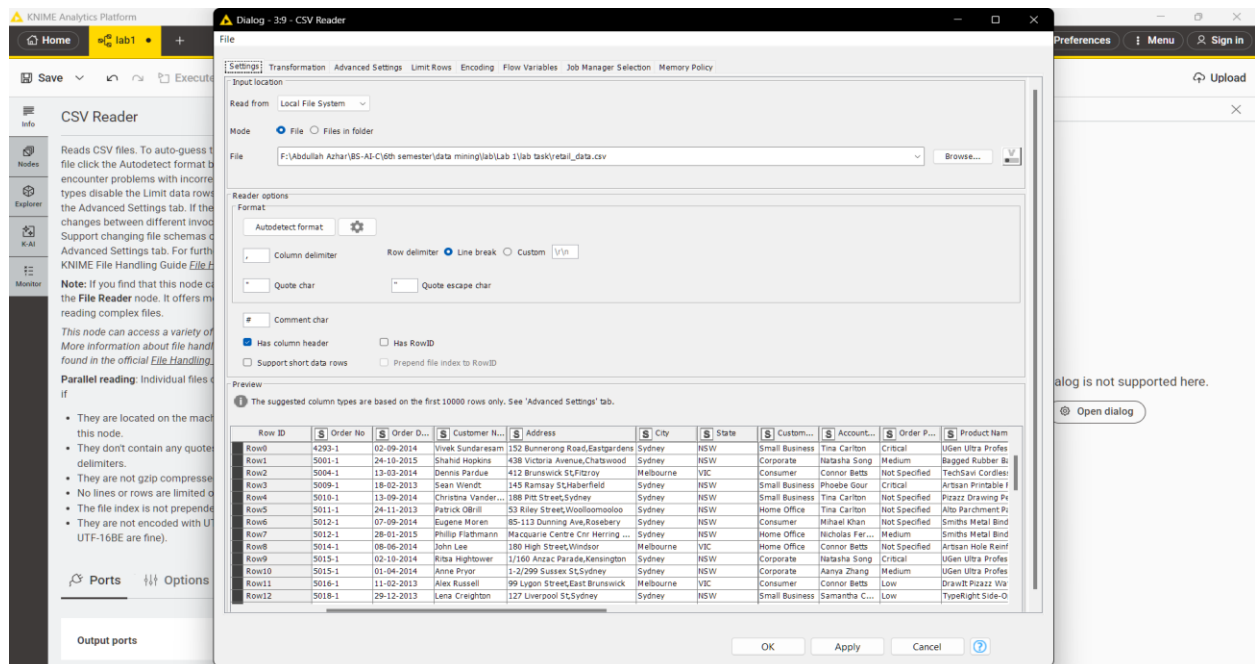
Step 1. Create a New Workflow

Launch KNIME Analytics Platform. From the menu, go to File ,New , New KNIME Workflow. Name the workflow Retail Sales Analysis and click Finish. This creates a blank canvas where all nodes will be placed and connected.



Step 2. Import the Dataset

Search for the **CSV Reader** node in the Node Repository panel on the left. Drag it onto the workflow canvas, then double-click it to open its settings. Click Browse and select the Retail Sales CSV file, then click OK. Right-click the node and select Execute. Once execution is complete, right-click again and choose View Table to confirm the data has loaded correctly.



Step 3. Verify Data Types

After loading, inspect the column types shown in the CSV Reader output. The Quantity column must be an Integer, Price must be a Double, Order Date must be a Date type, and Category must be a String. If any column is incorrectly typed — for instance, if Price is read as a String — apply the appropriate conversion node before proceeding. Use String to Number for numeric columns and String to Date&Time for the date column.

Rows: 5000 | Columns: 24

Product ...	Product C...	Product C...	Ship Mode	Ship Date	Cost Price	Retail Price	Profit Ma...	Order Qu...	Sub Total	Disco
Binder Posts	Office Supplies	Small Box	Regular Air	14-08-2015	\$3.50	\$5.74	\$2.24	31	\$213.34	0%
Artisan 479 Lab	Office Supplies	Wrap Bag	Regular Air	20-06-2016	\$1.59	\$2.61	\$1.02	23	\$66.54	5%
Laser DVD-RAM	Furniture	Small Pack	Regular Air	30-09-2016	\$20.18	\$35.41	\$15.23	19	\$929.40	9%
Smiths Colored	Office Supplies	Small Pack	Regular Air	18-12-2015	\$19.83	\$30.98	\$11.15	49	\$1,999.69	7%
Artisan Hi-Liter	Office Supplies	Small Pack	Regular Air	06-11-2016	\$2.98	\$5.84	\$2.86	35	\$115.40	2%
210 Trimline Ph	Office Supplies	Small Pack	Regular Air	15-05-2016	\$9.91	\$15.99	\$6.08	2	\$864.20	5%

Step 4. Generate Summary Statistics

Add a **Statistics** node to the canvas and connect it to the output port of the CSV Reader. Execute it and view the result. The output shows the mean, minimum, maximum, standard deviation, and sum for each numeric column. This step provides an immediate snapshot of data distribution and helps identify any anomalies before analysis begins.

KNIME Analytics Platform

Home lab1 + Preferences Menu Sign In

Save Execute Cancel Reset

Nodes

Search: stat

DB Query Read/Write +32

Statistics Statistics View Tree Ensemble Statistics

Crosstab Bar Chart Stacked Area Chart

DB Data Spec Extractor Example Data Reader Matrix Distance

DB Reader DB Query Reader DB Query

DB Joiner DB GroupBy DB Row Filter

Microsoft SQL Server Connector DB SQL Executor DB Connector

DB Column Filter SQLite Connector PostgreSQL Connector

MySQL Connector Oracle Connector DB Connection Closer

Statistics View

Decision Tree Predictor

CSV Reader

Statistics

Add comment

Find more resources via "Help"

Get quick access to resources like our self-paced courses, forum, cheat sheets, documentation, keyboard shortcuts and lots of examples on our Hub.

Nominal Values

Column filter

Manual Wildcard Regex Type

Search

Excludes

Includes

Order No

Order Date

Customer Name

Address

City

State

Discard Apply and Execute Apply

1: Statistics Table 2: Nominal Histogram Table 3: Occurrences Table Flow Variables

Rows: 1 Columns: 16

#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis	Overall su...	No. r
1	Order C	Order Quantity	1	50	26.483	14.392	207.126	-0.092	-1.207	132,389	1

Step 5. Calculate the Revenue Column

Add a **Math Formula** node and connect it to the CSV Reader. Inside the node configuration, create a new column named Revenue using the formula:

\$Quantity\$ * \$Price\$

Execute the node. Each row in the output now contains the revenue value for that individual transaction, which will serve as the base for all KPI calculations.

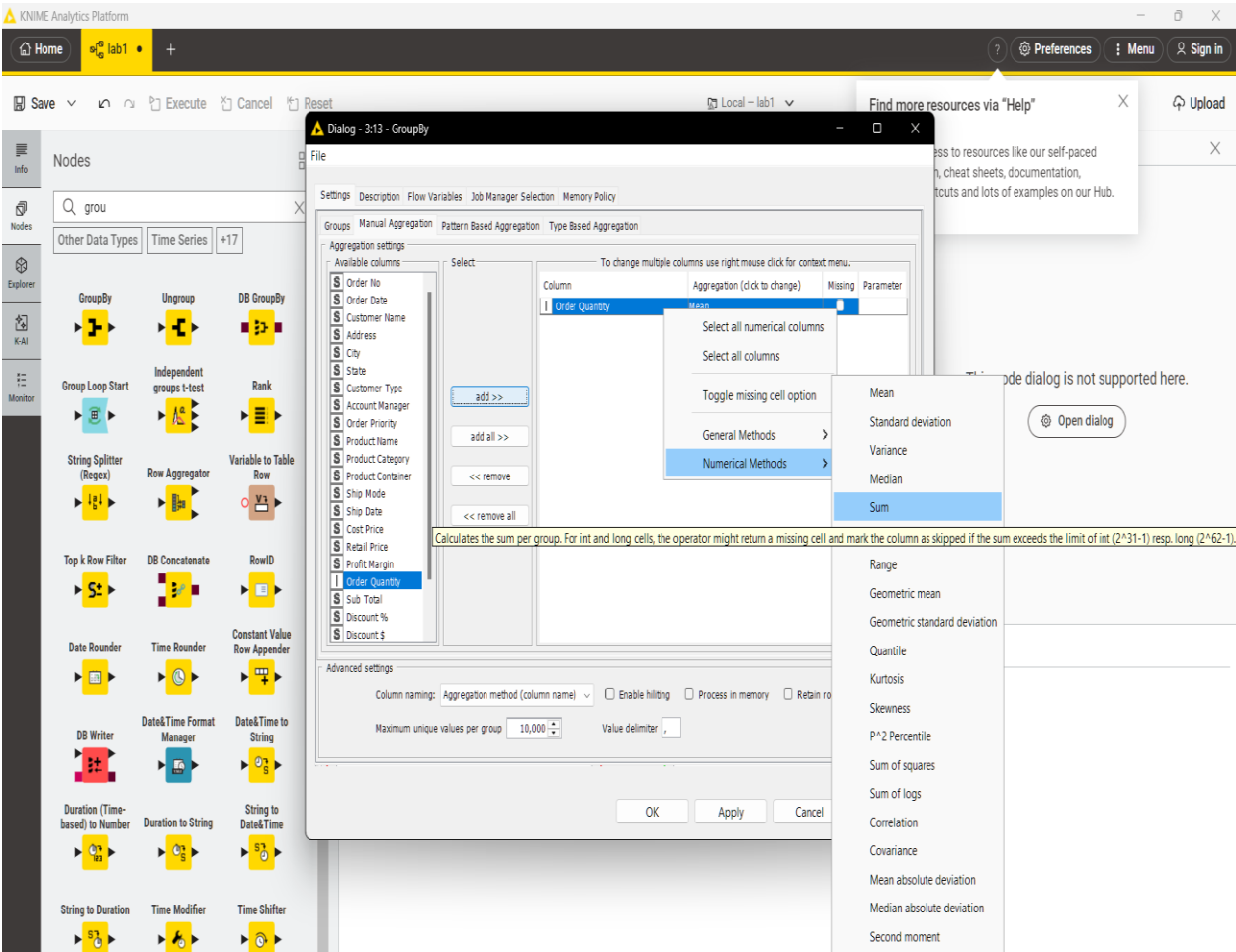
The screenshot displays the KNIME Analytics Platform interface. The main window shows a workflow with a CSV Reader node connected to a Math Formula node. The Math Formula node is selected, and its configuration dialog is open. The dialog has tabs for 'File', 'Math Expression', 'Flow Variables', 'Job Manager Selection', and 'Memory Policy'. The 'Math Expression' tab is active, showing a list of columns on the left and a list of functions on the right. The 'Column List' includes ROWINDEX, ROWCOUNT, and Order Quantity. The 'Function' list includes ROWCOUNT, ROWINDEX, pi, e, COL_MIN(col_name), COL_MAX(col_name), COL_MEAN(col_name), COL_MEDIAN(col_name), COL_SUM(col_name), COL_STDEV(col_name), COL_VAR(col_name), ln(x), log(x), log10(x), and exp(x). The 'Expression' field contains the formula: $1 \text{ } \$Order \text{ Quantity}\$ * \$Order \text{ Quantity}\$$. The 'Append Column' option is selected, and the 'Column Name' is set to 'Total'. The 'Convert to Int' option is unchecked. The 'OK' button is highlighted.

Below the dialog, a table of data is visible, showing columns for shipping, total, and multiplied values.

shipping	total	multiplied
1.18	\$4,291.55	529
1.70	\$46.91	64
1.18	\$82.58	2,025
1.19	\$730.92	256
2.78	\$125.97	2,401
7.28	\$189.43	2,025

KPI 1 Total Revenue

Add a GroupBy node with no grouping column selected. In the Manual Aggregation tab, select the order quantity column and set the aggregation method to Sum. Execute the node. The single output row shows the total revenue across all transactions.

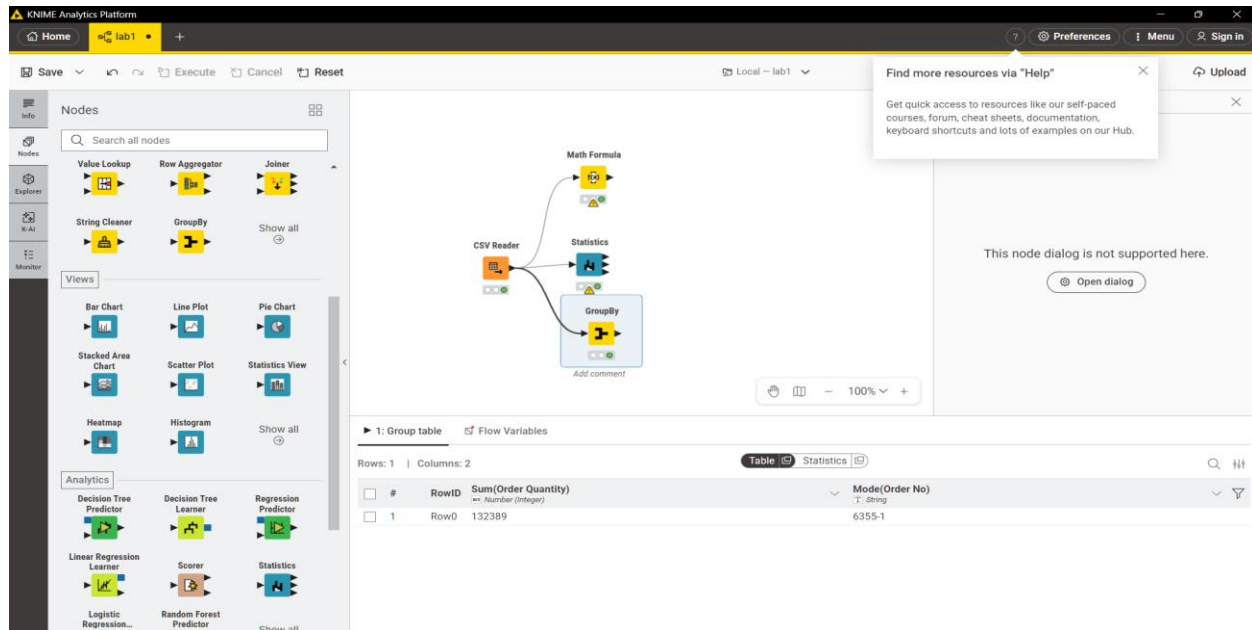


Rows: 1 | Columns: 1

	#	RowID	Sum(Order Quantity)
			123 Number (Integer)
	1	Row0	132389

KPI 2 Average Order quantity

Add a GroupBy node and group by Order ID, aggregating Revenue using Sum. This produces the total revenue per order. Connect a second GroupBy node with no grouping column, aggregating the result using Mean. The output is the average revenue per order.

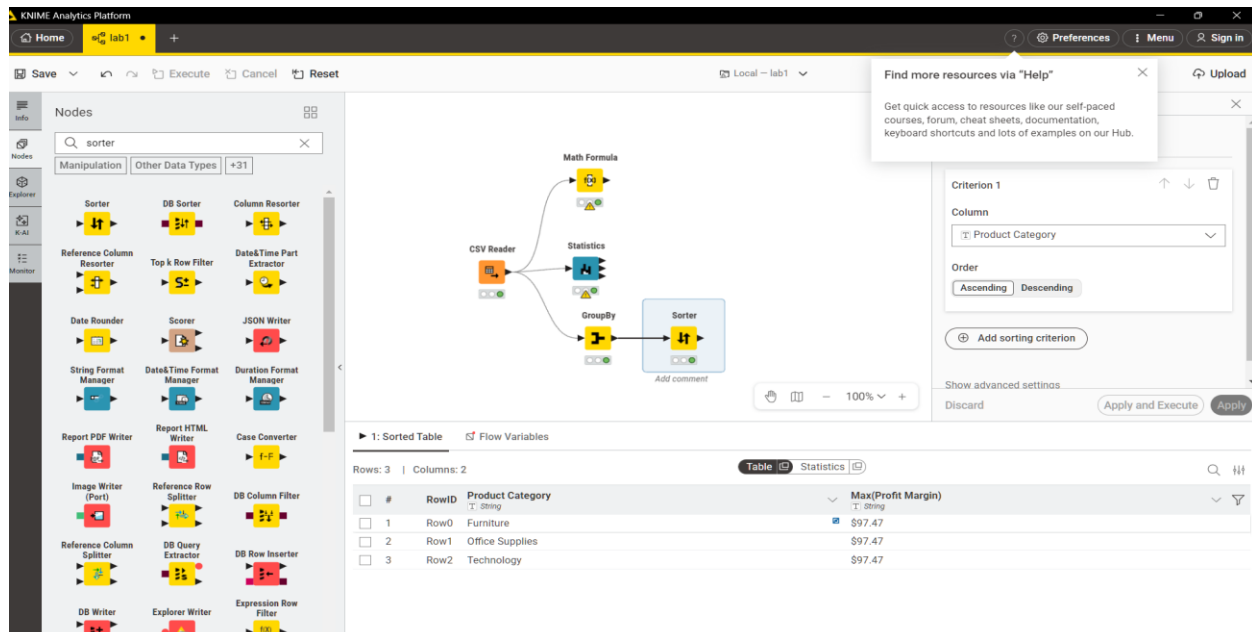


The screenshot shows the KNIME Analytics Platform interface. The workflow consists of a CSV Reader node connected to a Statistics node, which is then connected to a GroupBy node. The GroupBy node is configured to group by 'OrderID' and aggregate 'Revenue' using the 'Sum' function. The output table is displayed below the workflow.

#	RowID	Sum(Order Quantity)	Mode(Order No)
1	Row0	132389	6355-1

KPI 3 Category Revenue

Add a GroupBy node and group by the Category column, aggregating Revenue using Sum. Connect a Sorter node configured to sort by Revenue in descending order. The resulting table ranks each product category from highest to lowest earnings.



The screenshot shows the KNIME Analytics Platform interface. The workflow consists of a CSV Reader node connected to a Statistics node, which is then connected to a GroupBy node. The GroupBy node is configured to group by 'Product Category' and aggregate 'Revenue' using the 'Sum' function. The output table is displayed below the workflow.

#	RowID	Product Category	Max(Profit Margin)
1	Row0	Furniture	\$97.47
2	Row1	Office Supplies	\$97.47
3	Row2	Technology	\$97.47

KPI 4 Top Products

Add a GroupBy node and group by Product Name, aggregating Revenue using Sum. Connect a Sorter node sorted by Revenue in descending order. The top rows of the output identify the highest revenue-generating products.

The screenshot shows the KNIME Analytics Platform interface. On the left, the 'Nodes' panel lists various data manipulation and analysis nodes. The main workspace displays a workflow: a CSV Reader node feeds into a Statistics node, which then feeds into a GroupBy node. The GroupBy node is configured to group by 'Product Name' and aggregate 'Revenue' using the 'Sum' function. The output of the GroupBy node is connected to a Sorter node, which is configured to sort by 'Revenue' in descending order. The Sorter node's output is displayed in a table view showing the top 3 products: Row0 (Furniture) with 12 Colored Short Pencils, Row1 (Office Supplies) with UGen Ultra Professional Cordless Optical Suite, and Row2 (Technology) with Smiths Metal Binder Clips. A 'Find more resources via Help' tooltip is visible over the Sorter node configuration panel.

KPI Summary:

KPI	Method	KNIME Nodes Used
Total Revenue	Sum of (Quantity × Price)	Math Formula: GroupBy (Sum)
Average Order Value	Mean revenue per Order ID	GroupBy (Order ID) : GroupBy (Mean)
Category Revenue	Revenue grouped by Category	GroupBy (Category) : Sorter (Desc)
Top Products	Revenue grouped by Product Name	GroupBy (Product Name): Sorter (Desc)

Lab 2: CRISP-DM Methodology

Real World Problem

An e-commerce company is experiencing declining revenue and suspects a significant portion of its customer base is churning. The management wants to predict which customers are likely to stop purchasing so that targeted retention campaigns can be deployed proactively. This lab applies the CRISP-DM framework to structure the entire data mining process from business understanding through to evaluation.

Dataset

Customer Churn Dataset containing customer demographics, subscription details, usage patterns, support interactions, and a binary churn label indicating whether the customer left the service.

Practical Objectives

By the end of this lab, students will be able to apply all six CRISP-DM phases to a real churn prediction problem using Orange Data Mining. Students will document business objectives, explore and prepare the dataset, build and evaluate a predictive model, and produce a deployment plan.

Procedure:

Step 1. Business Understanding

Define the business goal: predict customer churn. Identify the success criteria: a model achieving at least 80% accuracy and an ROC-AUC score above 0.75. Document what churn means in the context of this company and list the data sources available.

Customer churn refers to the phenomenon where customers discontinue their relationship or subscription with a company or service provider. It represents the rate at which customers stop using a company's products or services within a specific period. Churn is an important metric for businesses as it directly impacts revenue, growth, and customer retention.

In the context of the Churn dataset, the churn label indicates whether a customer has churned or not. A churned customer is one who has decided to discontinue their subscription or usage of the company's services. On the other hand, a non-churned customer is one who continues to remain engaged and retains their relationship with the company.

Understanding customer churn is crucial for businesses to identify patterns, factors, and indicators that contribute to customer attrition. By analyzing churn behavior and its associated features, companies can develop strategies to retain existing customers, improve customer satisfaction, and reduce customer turnover. Predictive modeling techniques can also be applied to forecast and proactively address potential churn, enabling companies to take proactive measures to retain at-risk customers.

customer_churn_dataset-testing-master.csv (3.28 MB)

📄 🗂️ ➔

Detail Compact Column

10 of 12 columns ▾

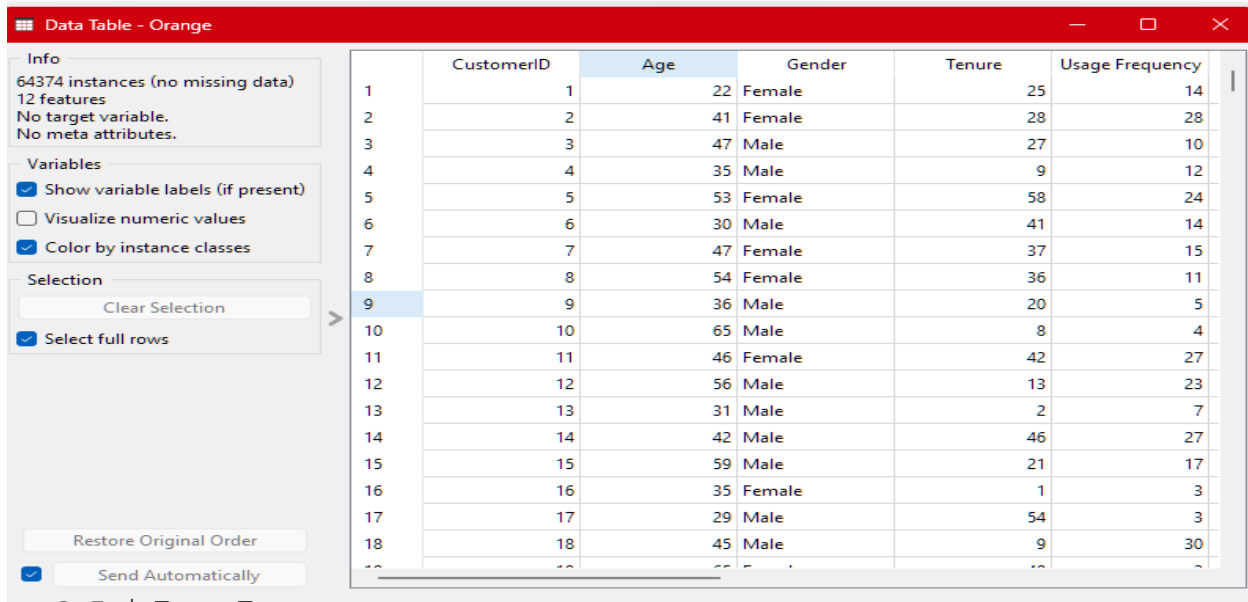
About this file

✎ Suggest Edits

The testing file for the churn dataset consists of 64,374 customer records and serves as a separate dataset for evaluating the performance and generalization capabilities of trained churn prediction models. Each record in the testing file corresponds to a customer and contains the same set of features as the training file, such as age, gender, tenure, usage frequency, support calls, payment delay, subscription type, contract length, total spend, and last interaction. However, the churn labels are not included in the testing file as they are used for assessing the accuracy and effectiveness of the churn prediction models. The testing file allows businesses to assess the predictive power of their trained models on unseen data and gain insights into how well the models generalize to new customers. By analyzing the model's performance on the testing file, businesses can gauge the effectiveness of their churn prediction strategies and make informed decisions to optimize customer retention efforts.

Step 2. Data Understanding

Open Orange Data Mining. Load the Customer Churn CSV using the File widget. Connect a Data Table widget to inspect rows and columns. Use the Feature Statistics widget to view distributions, missing value counts, and class balance. Record the churn rate as a baseline metric.



Data Table - Orange

Info
64374 instances (no missing data)
12 features
No target variable.
No meta attributes.

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

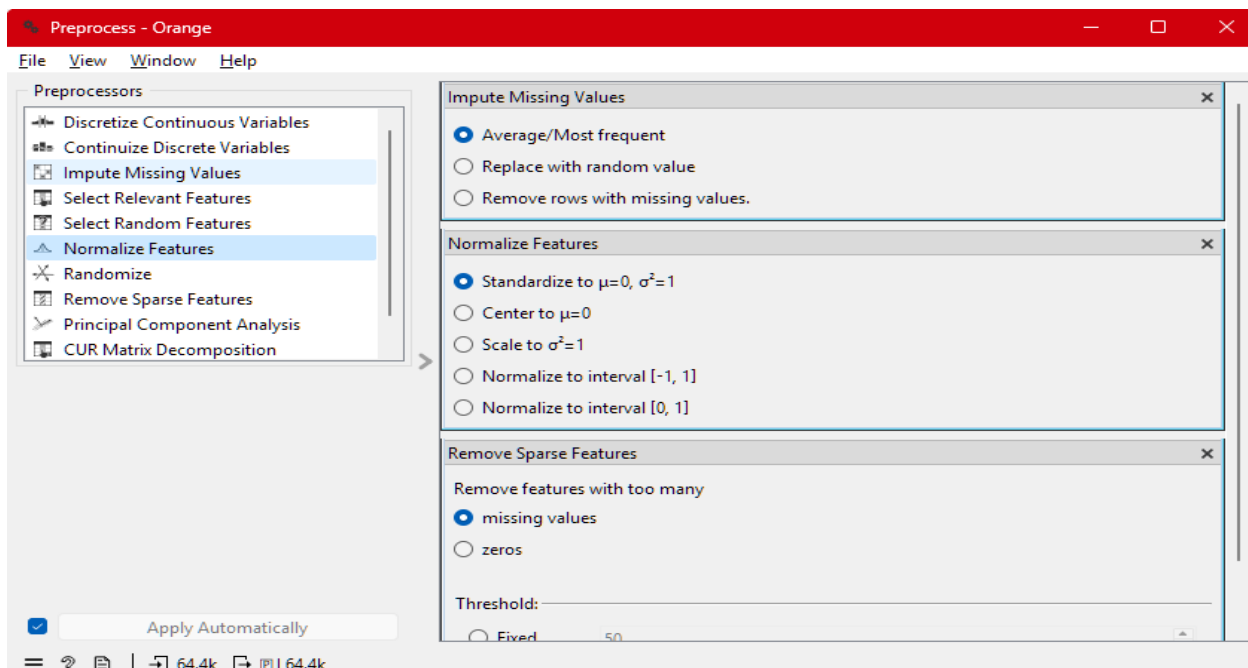
Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

	CustomerID	Age	Gender	Tenure	Usage Frequency
1	1	22	Female	25	14
2	2	41	Female	28	28
3	3	47	Male	27	10
4	4	35	Male	9	12
5	5	53	Female	58	24
6	6	30	Male	41	14
7	7	47	Female	37	15
8	8	54	Female	36	11
9	9	36	Male	20	5
10	10	65	Male	8	4
11	11	46	Female	42	27
12	12	56	Male	13	23
13	13	31	Male	2	7
14	14	42	Male	46	27
15	15	59	Male	21	17
16	16	35	Female	1	3
17	17	29	Male	54	3
18	18	45	Male	9	30

Step 3. Data Preparation

Use the Preprocess widget to handle missing values (impute with mean for numeric, most frequent for categorical), encode categorical features using the Continuize widget, and normalize all numeric attributes. Remove any identifier columns such as CustomerID that carry no predictive value.



Preprocess - Orange

File View Window Help

Preprocessors

- Discretize Continuous Variables
- Continuize Discrete Variables
- Impute Missing Values
- Select Relevant Features
- Select Random Features
- Normalize Features
- Randomize
- Remove Sparse Features
- Principal Component Analysis
- CUR Matrix Decomposition

Apply Automatically

64.4k 64.4k

Impute Missing Values

- ☒ Average/Most frequent
- ☐ Replace with random value
- ☐ Remove rows with missing values.

Normalize Features

- ☒ Standardize to $\mu=0, \sigma^2=1$
- ☐ Center to $\mu=0$
- ☐ Scale to $\sigma^2=1$
- ☐ Normalize to interval $[-1, 1]$
- ☐ Normalize to interval $[0, 1]$

Remove Sparse Features

Remove features with too many

- ☒ missing values
- ☐ zeros

Threshold:

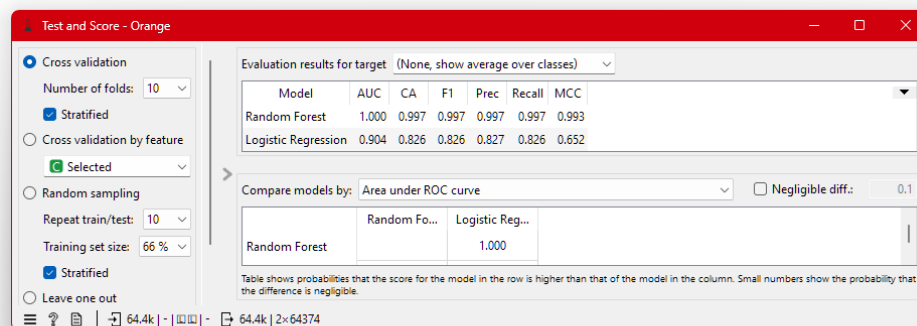
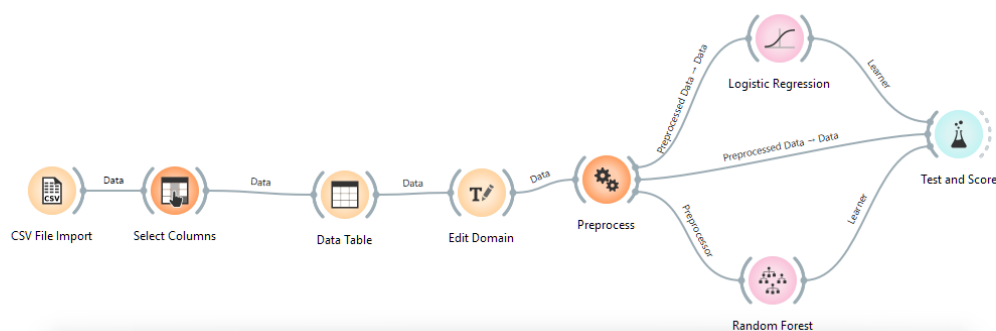
Step 4. Modelling

Add a Logistic Regression learner and a Random Forest learner to the canvas. Connect both to the preprocessed data. Link them to the Test and Score widget configured with 10-fold cross-validation and the Churn column set as the target variable.



Step 5. Evaluation

View the Test and Score results. Record Accuracy, Precision, Recall, F1, and ROC-AUC for both models. Connect the ROC Analysis widget to visualise the curves. Compare both models against the success criteria defined in Step 1 and select the better performer.



KPI Summary

Evaluation results for target (None, show average over classes) ▼

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest	1.000	0.997	0.997	0.997	0.997	0.993
Logistic Regression	0.904	0.826	0.826	0.827	0.826	0.652

KPI / Metric	Definition	Random Forest	Logistic Regression	Orange Node
Model Accuracy (CA)	Percentage of total correct guesses	99.7%	82.6%	Test and Score
ROC-AUC Score	Ability to distinguish between classes	1.000	0.904	Test and Score
F1 Score	Harmonic mean of Precision and Recall	0.997	0.826	Test and Score
Precision	Accuracy of positive (Churn) predictions	0.997	0.827	Test and Score
Recall	Ability to find all actual churners	0.997	0.826	Test and Score

Lab 3: Data Cleaning

Real World Problem

A hospital is preparing patient records collected from multiple departments for a disease prediction study. The raw data contains missing diagnostic values, duplicate patient entries from system migrations, and noisy sensor readings. Before any analysis can proceed, the data must be cleaned to a standard suitable for clinical machine learning.

Dataset

Healthcare Dataset patient records including age, gender, blood pressure, glucose, BMI, diagnosis codes, and admission dates with intentionally introduced missing values, duplicates, and outliers.

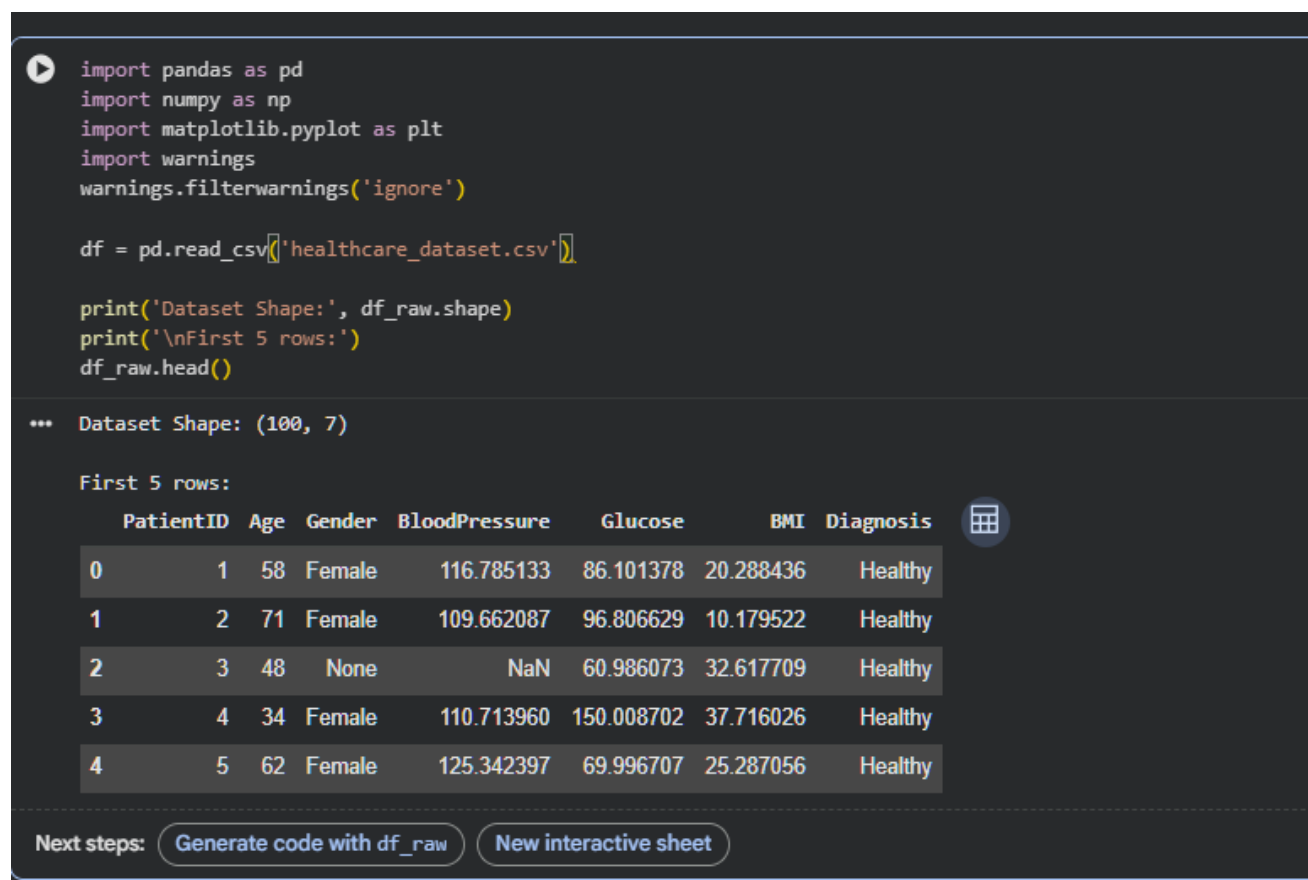
Practical Objectives

Students will identify and handle missing values using multiple strategies, detect and remove duplicate records, apply noise filtering using smoothing techniques, and calculate a data quality score before and after cleaning using RapidMiner Studio and Python.

Procedure

Step 1. Load and Profile the Data

Use the Read CSV operator to import the healthcare dataset. Connect a Retrieve operator to the process and run it. View the data in the Results panel. Note the total row count, column types, and any columns flagged with missing indicators.



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('healthcare_dataset.csv')

print('Dataset Shape:', df.shape)
print('\nFirst 5 rows:')
df.head()
```

Dataset Shape: (100, 7)

First 5 rows:

	PatientID	Age	Gender	BloodPressure	Glucose	BMI	Diagnosis
0	1	58	Female	116.785133	86.101378	20.288436	Healthy
1	2	71	Female	109.662087	96.806629	10.179522	Healthy
2	3	48	None	NaN	60.986073	32.617709	Healthy
3	4	34	Female	110.713960	150.008702	37.716026	Healthy
4	5	62	Female	125.342397	69.996707	25.287056	Healthy

Next steps: [Generate code with df_raw](#) [New interactive sheet](#)

Step 1: Load and Profile the Data

```
# Profile the dataset
print('=== DATASET PROFILE ===')
print(f'Total Rows      : {df_raw.shape[0]}')
print(f'Total Columns    : {df_raw.shape[1]}')
print(f'\nData Types:')
print(df_raw.dtypes)
print(f'\nMissing Values per Column:')
print(df_raw.isnull().sum())
print(f'\nTotal Missing Cells : {df_raw.isnull().sum().sum()}')
print(f'Total Duplicate Rows: {df_raw.duplicated(subset="PatientID").sum()}')
```

```
=== DATASET PROFILE ===
Total Rows      : 100
Total Columns    : 7

Data Types:
PatientID      int64
Age            int64
Gender         object
BloodPressure  float64
Glucose        float64
BMI            float64
Diagnosis      object
dtype: object

Missing Values per Column:
PatientID      0
Age            0
Gender         9
BloodPressure  5
Glucose       10
BMI           8
Diagnosis      7
dtype: int64

Total Missing Cells : 39
Total Duplicate Rows: 10
```

Step 2. Handle Missing Values

Add the Replace Missing Values operator. For numeric columns such as Glucose and BMI, set the replacement method to Average. For categorical columns such as Diagnosis, set it to Most Frequent Value. For columns where more than 40% of values are missing, use the Remove Useless Attributes operator to drop them entirely.

Step 2: Handle Missing Values

```
df_clean = df_raw.copy()

# --- Numeric columns: fill with mean ---
numeric_cols = ['Glucose', 'BMI', 'BloodPressure']
for col in numeric_cols:
    before = df_clean[col].isnull().sum()
    df_clean[col].fillna(df_clean[col].mean(), inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing → filled with mean ({df_raw[col].mean():.2f}) → {after} missing')
```

```
# --- Categorical columns: fill with most frequent value ---
cat_cols = ['Gender', 'Diagnosis']
for col in cat_cols:
    before = df_clean[col].isnull().sum()
    most_freq = df_clean[col].mode()[0]
    df_clean[col].fillna(most_freq, inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing → filled with "{most_freq}" → {after} missing')
```

```
print(f'\nTotal missing after handling: {df_clean.isnull().sum().sum()}')
```

```
Glucose: 10 missing → filled with mean (101.08) → 0 missing
BMI: 8 missing → filled with mean (26.49) → 0 missing
BloodPressure: 5 missing → filled with mean (118.62) → 0 missing
Gender: 9 missing → filled with "Male" → 0 missing
Diagnosis: 7 missing → filled with "Healthy" → 0 missing

Total missing after handling: 0
```


Step 3. Remove Duplicates

Add the Remove Duplicates operator. Configure it to detect duplicate rows based on PatientID. Keep only the first occurrence. Record how many rows were removed. Then switch to Python and validate the result using pandas: `df.duplicated().sum()` before and after.

Step 3: Remove Duplicate Records

```
# Check duplicates BEFORE
dups_before = df_clean.duplicated(subset='PatientID').sum()
print(f'Duplicate rows BEFORE removal: {dups_before}')
print(f'Total rows BEFORE          : {len(df_clean)}')

# Remove duplicates – keep first occurrence
df_clean = df_clean.drop_duplicates(subset='PatientID', keep='first')

dups_after = df_clean.duplicated(subset='PatientID').sum()
print(f'\nDuplicate rows AFTER removal : {dups_after}')
print(f'Total rows AFTER          : {len(df_clean)}')
print(f'Rows removed              : {dups_before}')
```

```
Duplicate rows BEFORE removal: 10
Total rows BEFORE          : 100
```

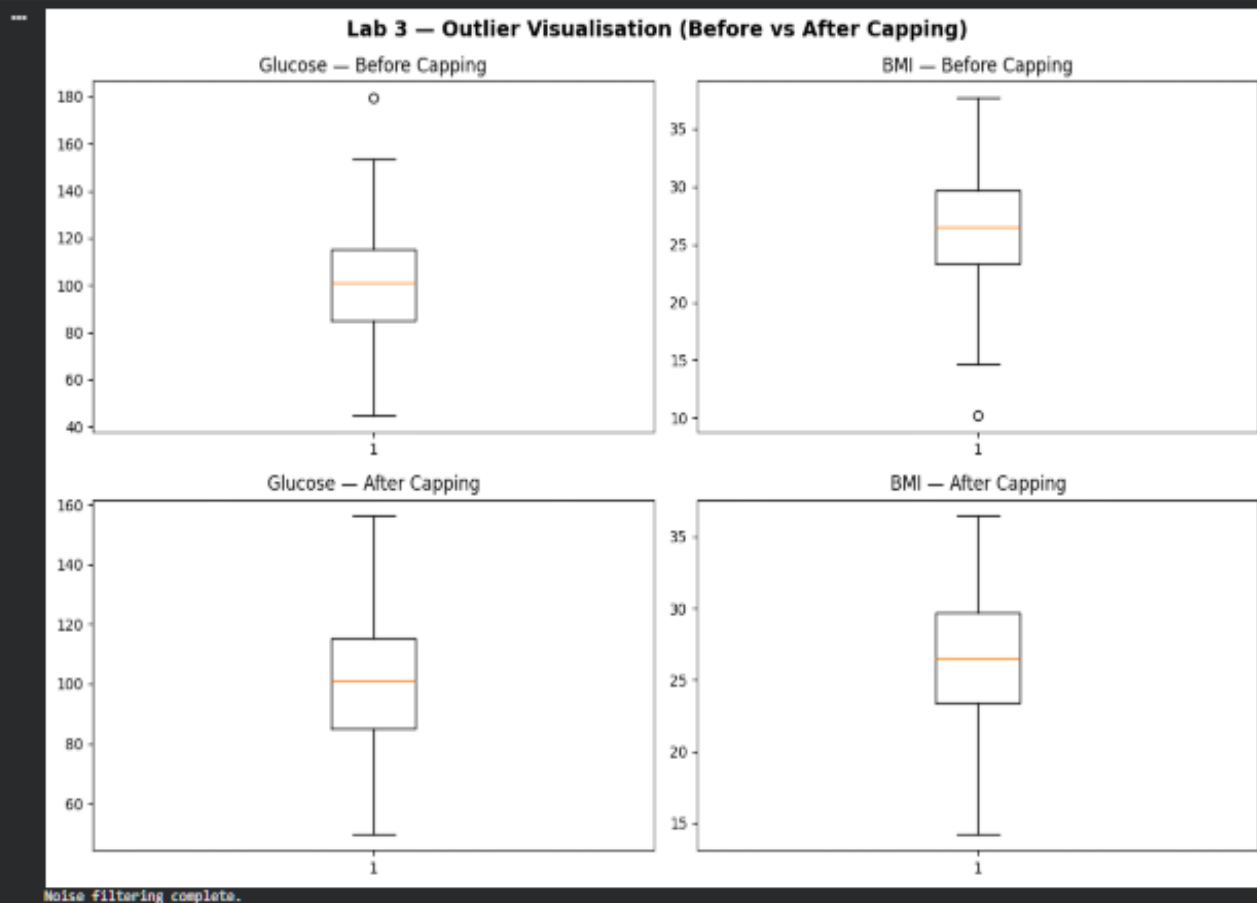
```
Duplicate rows AFTER removal : 0
Total rows AFTER          : 90
Rows removed              : 10
```

Step 4. Noise Filtering in Python

Export the cleaned data from RapidMiner as CSV. Load it in Python using pandas. Apply a rolling mean smoothing with a window of 3 on the BloodPressure column to reduce sensor noise. Use box plots (via matplotlib) to visualise outliers in Glucose and BMI before and after filtering. Cap extreme values at the 1st and 99th percentiles using np.clip.

Step 4: Noise Filtering

```
df_clean['BloodPressure_Smoothed'] = df_clean['BloodPressure'].rolling(window=3, min_periods=1).mean()
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Lab 3 - Outlier Visualisation (Before vs After Capping)', fontsize=14, fontweight='bold')
axes[0, 0].boxplot(df_clean['Glucose'].dropna())
axes[0, 0].set_title('Glucose - Before Capping')
axes[0, 1].boxplot(df_clean['BMI'].dropna())
axes[0, 1].set_title('BMI - Before Capping')
for col in ['Glucose', 'BMI', 'BloodPressure']:
    low = df_clean[col].quantile(0.01)
    high = df_clean[col].quantile(0.99)
    df_clean[col] = np.clip(df_clean[col], low, high)
axes[1, 0].boxplot(df_clean['Glucose'].dropna())
axes[1, 0].set_title('Glucose - After Capping')
axes[1, 1].boxplot(df_clean['BMI'].dropna())
axes[1, 1].set_title('BMI - After Capping')
plt.tight_layout()
plt.show()
print('Noise filtering complete.')
```



KPI Summary

```
total_cells = df_raw.shape[0] * df_raw.shape[1]
missing_before = df_raw.isnull().sum().sum()
missing_after = df_clean.isnull().sum().sum()
dups_removed = dups_before
total_before = df_raw.shape[0]
valid_cells = total_cells - missing_after

pct_missing_reduced = ((missing_before - missing_after) / missing_before) * 100 if missing_before > 0 else 100
pct_dup_removed = (dups_removed / total_before) * 100
data_quality_score = (valid_cells / total_cells) * 100

print('=' * 55)
print('          KPI SUMMARY – LAB 3: DATA CLEANING')
print('=' * 55)
print(f'% Missing Values Reduced : {pct_missing_reduced:.2f}%')
print(f'Duplicate Removal %      : {pct_dup_removed:.2f}%')
print(f'Data Quality Score       : {data_quality_score:.2f}%')
print('=' * 55)
print('Target: Data Quality Score > 95%')
print('Status:', '✅ PASSED' if data_quality_score > 95 else '❌ NEEDS MORE CLEANING')
```

```
=====
          KPI SUMMARY – LAB 3: DATA CLEANING
=====
% Missing Values Reduced : 100.00%
Duplicate Removal %      : 10.00%
Data Quality Score       : 100.00%
=====
Target: Data Quality Score > 95%
Status: ✅ PASSED
```

KPI	Method	Results
% Missing Values Reduced	$((\text{Missing Before} - \text{Missing After}) \div \text{Missing Before}) \times 100$	100%
Duplicate Removal %	$(\text{Duplicates Removed} \div \text{Total Records Before}) \times 100$	10%
Data Quality Score	$(\text{Valid Cells} \div \text{Total Cells}) \times 100$	Greater than 95%

Lab 4: Data Preprocessing

Real World Problem

A bank is preparing a loan applicant dataset for a risk prediction model. Raw financial data contains attributes with very different scales (income vs. credit score), skewed distributions, and continuous variables that need to be converted into meaningful categories for rule-based systems. Preprocessing must be applied before model training can begin.

Dataset

Loan Prediction Dataset containing applicant income, loan amount, credit history, employment status, loan term, and a binary loan approval label.

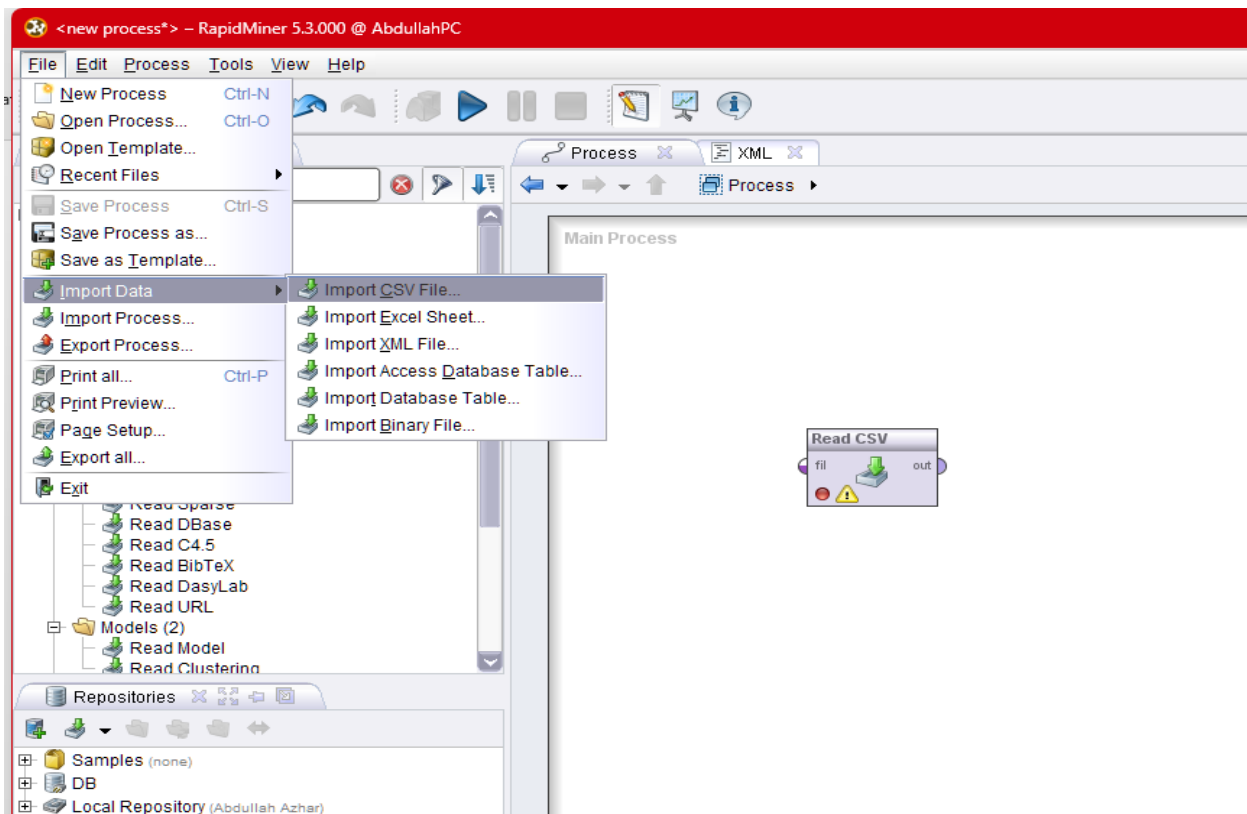
Practical Objectives

Students will apply min-max normalization, Z-score standardisation, and equal-width discretization on the loan dataset using RapidMiner Studio. Students will observe how each transformation affects the data distribution and record summary statistics before and after each step.

Procedure:

Step 1. Load Data and Generate Baseline Statistics

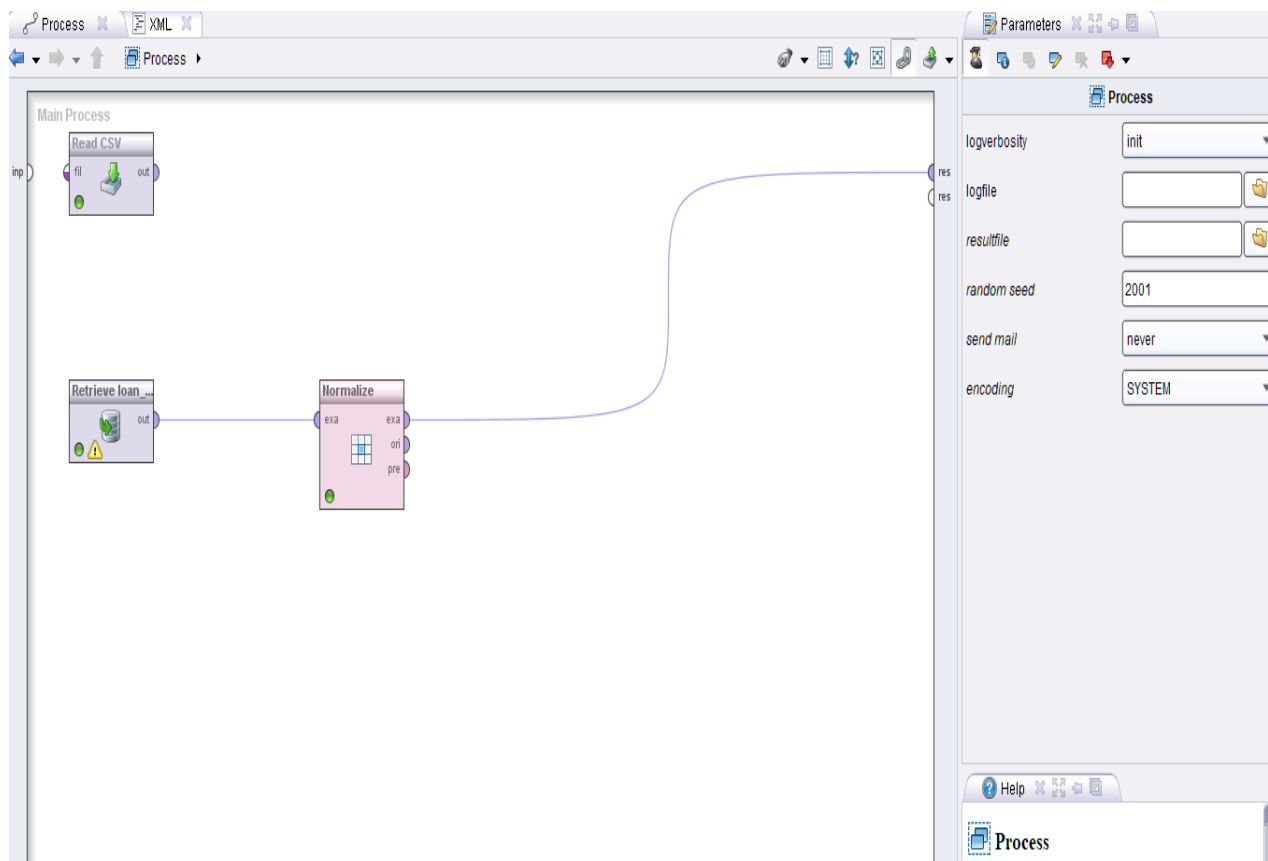
In rapid miner Import the Loan Prediction CSV using the Read CSV operator. Add a Statistics operator and execute the process to capture the mean, variance, minimum, and maximum for ApplicantIncome and LoanAmount. Save these baseline values for comparison after preprocessing.



Step 2. Apply Min-Max Normalization

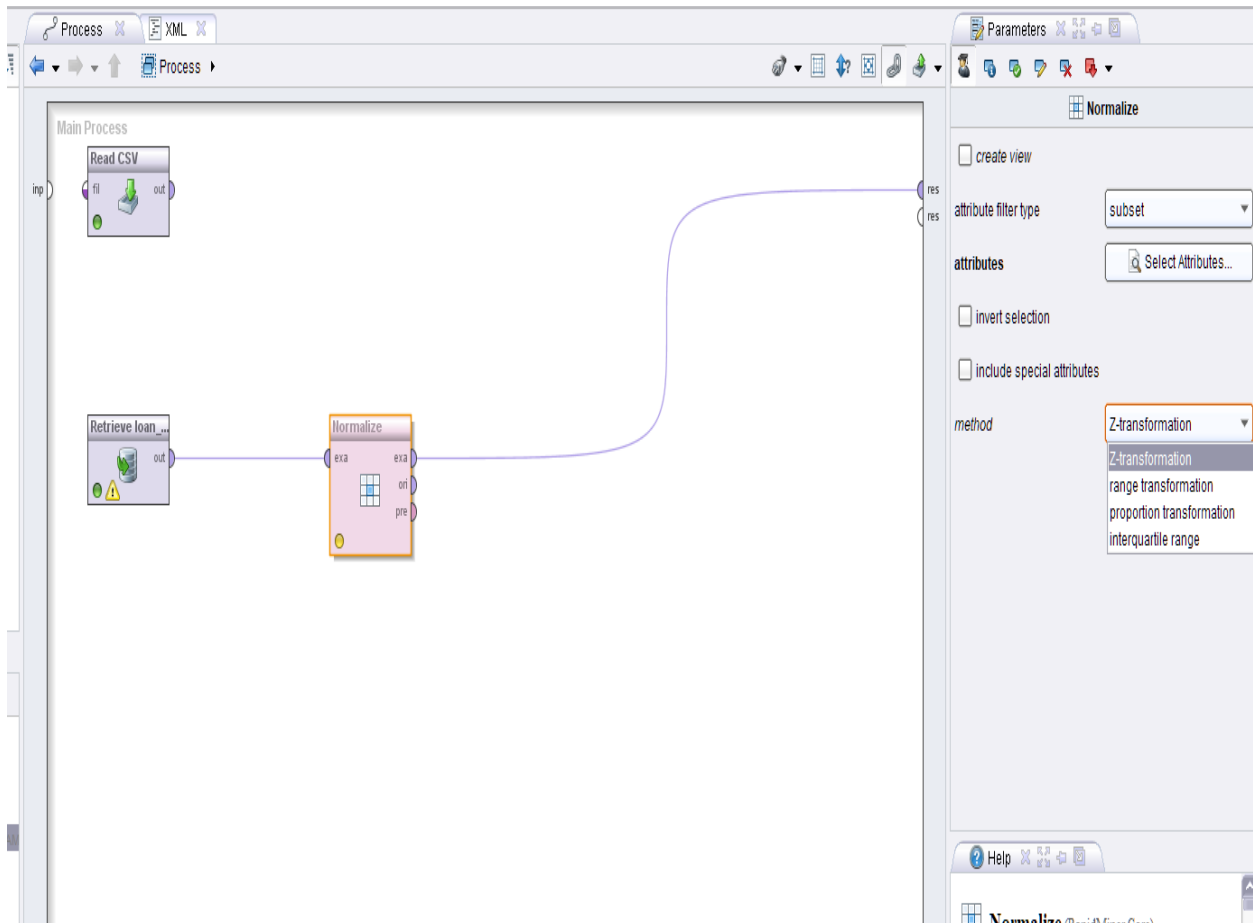
Add the Normalize operator. Set the method to Range Transformation with a minimum of 0 and maximum of 1. Apply it to ApplicantIncome and LoanAmount. Execute and view the transformed values. Verify that all values now fall within [0, 1] and record the updated statistics.

Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	0.046	0.344
2	1	0.036	0.457
3	1	0.051	0.065
4	1	0.015	0.297
5	1	0.049	0.331
6	1	0.126	0.252
7	0	0.047	0.111



Step 3. Apply Z-Score Standardisation

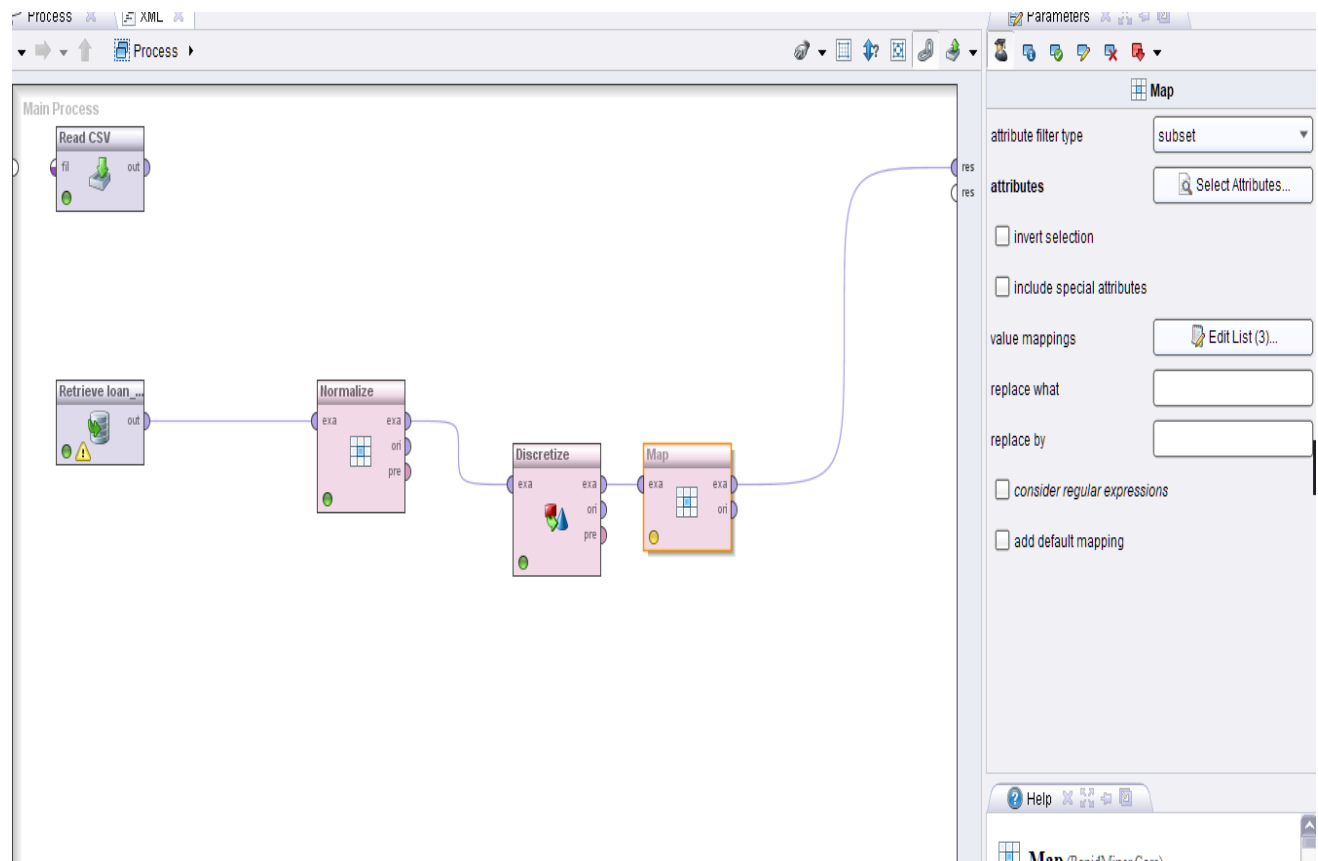
Replace the Normalize operator with a fresh one and set the method to Standardisation (Z-transformation). Apply to the same columns. Execute and verify the resulting mean is approximately 0 and standard deviation approximately 1. Compare this output against the min-max result and note the practical differences.



ExampleSet (20000 examples, 1 special attribute, 211 attributes)			
Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	-0.674	0.238
2	1	-0.815	0.876
3	1	-0.606	-1.336
4	1	-1.105	-0.022
5	1	-0.636	0.165
6	1	0.419	-0.280
7	0	-0.659	-1.072
8	1	0.316	0.001
9	1	0.676	-1.700

Step 4. Discretization

Add the Discretize operator. Set the method to Equal Width with 3 bins for ApplicantIncome. Label the bins as Low, Medium, and High. Execute and view the new categorical column. Observe how the continuous income variable has been transformed into an ordinal attribute suitable for rule-based classifiers.



sampleSet (20000 examples, 1 special attribute, 21 regular attributes)

row No.	loan_paid_off	annual_income	loan_amount	loan_status
1	1	low	0.238	59
2	1	low	0.876	72
3	1	low	-1.336	49
4	1	low	-0.022	39
5	1	low	0.165	69
6	1	low	-0.280	29
7	0	low	-1.072	49
8	1	low	0.001	59
9	1	low	-1.700	39

KPI Summary:

KPI	Method	Results
Variance Reduction	Compare variance before vs. after normalization	Before: 821,887,460.35\$ After: \$0.0053\$
Distribution Improvement	Compare skewness before vs. after standardization	Skewness Before: \$2.33\$ Skewness After: \$2.33\$
Income Category Distribution	Count of records in each discretized income bin	Low: \$19,736\$ records Medium: \$250\$ records High: \$14\$ records

Lab 5: Market Basket Analysis

Real World Problem

A supermarket chain wants to design a cross-selling strategy by understanding which products customers frequently purchase together. The business goal is to identify strong product associations from transaction data so that shelf layouts, promotions, and bundle offers can be optimised to increase average basket size.

Dataset

Grocery Transactions Dataset — each row represents a single transaction with a Transaction ID and a list of purchased items. Suitable for manual association rule calculation and Python validation.

Practical Objectives

Students will manually calculate Support, Confidence, and Lift for selected item pairs using a small transaction sample, then validate those calculations programmatically using Python with the mlxtend library, and interpret the strongest rules for business cross-selling recommendations.

Procedure:

Step 1. Manual Calculation

Take the first 10 transactions from the dataset. For each unique item pair, count the number of transactions containing both items. Apply the formulas: $\text{Support}(A \rightarrow B) = \text{Transactions}(A \cap B) / \text{Total Transactions}$; $\text{Confidence}(A \rightarrow B) = \text{Transactions}(A \cap B) / \text{Transactions}(A)$; $\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$. Record results in a table.

Step 0: Import Libraries & Sample Dataset

```
import pandas as pd
import numpy as np
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import warnings
warnings.filterwarnings('ignore')
df = pd.read_csv('grocery_transactions.csv')
transactions = df.groupby('TransactionID')['Item'].apply(list).tolist()

print(f'Total transactions: {len(transactions)}')
print('Sample transactions (first 5):')
for i, t in enumerate(transactions[:5]):
    print(f'  T[{i+1}]: {t}')
```

```
Total transactions: 28
Sample transactions (first 5):
T1: ['Milk', 'Bread', 'Butter']
T2: ['Milk', 'Bread']
T3: ['Milk', 'Eggs']
T4: ['Bread', 'Butter', 'Eggs']
T5: ['Milk', 'Butter']
```

Step 1: Manual Calculation of Support, Confidence, Lift

```
# Manual calculation using first 10 transactions
sample = transactions[:10]
total = len(sample)

def support(item_set, txns):
    count = sum(1 for t in txns if all(i in t for i in item_set))
    return count / len(txns)

def confidence(A, B, txns):
    sup_AB = support(A + B, txns)
    sup_A = support(A, txns)
    return sup_AB / sup_A if sup_A > 0 else 0

def lift(A, B, txns):
    conf_AB = confidence(A, B, txns)
    sup_B = support(B, txns)
    return conf_AB / sup_B if sup_B > 0 else 0

# Item pairs to check
pairs = [
    (['Milk'], ['Bread']),
    (['Milk'], ['Butter']),
    (['Bread'], ['Butter']),
    (['Milk'], ['Eggs']),
    (['Bread'], ['Eggs']),
]

print('--- MANUAL CALCULATION (first 10 transactions) ---')
print(f'{"Rule":<25} {"Support":>10} {"Confidence":>12} {"Lift":>8}')
print('-' * 60)
for A, B in pairs:
    sup = support(A + B, sample)
    conf = confidence(A, B, sample)
    lift = lift(A, B, sample)
    rule = f'{A[0]} -> {B[0]}'
    print(f'{rule:<25} {sup:>10.4f} {conf:>12.4f} {lift:>8.4f}')
```

```
--- MANUAL CALCULATION (first 10 transactions) ---
Rule                               Support  Confidence  Lift
-----
Milk -> Bread                      0.4000      0.6667  0.9524
Milk -> Butter                     0.3000      0.5000  0.8333
Bread -> Butter                    0.4000      0.5714  0.9524
Milk -> Eggs                      0.3000      0.5000  0.8333
Bread -> Eggs                     0.4000      0.5714  0.9524
```

Step 2. Python Validation with mlxtend

Load the full transaction dataset in Python. Use pandas to encode it as a one-hot boolean DataFrame. Apply TransactionEncoder from mlxtend.preprocessing. Run the apriori function with min_support=0.05. Then call association_rules with metric="lift" and min_threshold=1.0. Sort the output by lift descending and display the top 10 rules. Compare these results against your manual calculations from Step 1.

Step 2: Python Validation with mlxtend (Full Dataset)

```
# Encode transactions
te = TransactionEncoder()
te_array = te.fit_transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

# Run Apriori
frequent_itemsets = apriori(df_encoded, min_support=0.05, use_colnames=True)
rules = association_rules(frequent_itemsets, metric='lift', min_threshold=1.0)
rules = rules.sort_values('lift', ascending=False).reset_index(drop=True)

print(f'Total frequent itemsets: {len(frequent_itemsets)}')
print(f'Total association rules: {len(rules)}')
print(f'\nTop 10 Rules by Lift:')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(10).to_string())
```

Total frequent itemsets: 26
Total association rules: 8

Top 10 Rules by Lift:

	antecedents	consequents	support	confidence	lift
0	(Milk, Butter, Eggs)	(Bread)	0.05	1.000000	1.818182
1	(Bread)	(Milk, Butter, Eggs)	0.05	0.090909	1.818182
2	(Bread)	(Milk, Eggs)	0.15	0.272727	1.090909
3	(Milk, Eggs)	(Bread)	0.15	0.600000	1.090909
4	(Milk, Eggs)	(Butter, Bread)	0.05	0.200000	1.000000
5	(Butter, Bread)	(Milk, Eggs)	0.05	0.250000	1.000000
6	(Milk, Eggs)	(Bread, Cheese)	0.05	0.200000	1.000000
7	(Bread, Cheese)	(Milk, Eggs)	0.05	0.250000	1.000000

Step 3. Business Interpretation

Select the top 3 rules by Lift value. For each rule, write a one-sentence business recommendation explaining how the supermarket could act on this association (e.g., shelf placement, bundled discounts, or targeted promotions). Rules with Lift > 2 indicate strong cross-sell potential and should be prioritised.

Step 3: Business Interpretation

```
# Top 3 rules by Lift
top3 = rules.head(3)

print('=== TOP 3 RULES - BUSINESS RECOMMENDATIONS ===')
for i, row in top3.iterrows():
    ant = ', '.join(list(row['antecedents']))
    con = ', '.join(list(row['consequents']))
    print(f'\nRule {i+1}: {ant} → {con}')
    print(f'  Support   : {row["support"]:.4f}')
    print(f'  Confidence : {row["confidence"]:.4f}')
    print(f'  Lift       : {row["lift"]:.4f}')
    print(f'  Business Tip: Place "{con}" near "{ant}" in the store to boost cross-sales.')

...

=== TOP 3 RULES - BUSINESS RECOMMENDATIONS ===

Rule 1: Milk, Butter, Eggs → Bread
  Support   : 0.0500
  Confidence: 1.0000
  Lift       : 1.8182
  Business Tip: Place "Bread" near "Milk, Butter, Eggs" in the store to boost cross-sales.

Rule 2: Bread → Milk, Butter, Eggs
  Support   : 0.0500
  Confidence: 0.0909
  Lift       : 1.8182
  Business Tip: Place "Milk, Butter, Eggs" near "Bread" in the store to boost cross-sales.

Rule 3: Bread → Milk, Eggs
  Support   : 0.1500
  Confidence: 0.2727
  Lift       : 1.0909
  Business Tip: Place "Milk, Eggs" near "Bread" in the store to boost cross-sales.
```

KPI Summary:

```
KPI Summary — Lab 5

high_lift_rules = rules[rules['lift'] > 2]
cross_sell_score = rules.head(10)['lift'].mean()

print('=' * 55)
print('    KPI SUMMARY — LAB 5: MARKET BASKET ANALYSIS')
print('=' * 55)
print(f'Number of High Lift Rules (Lift > 2) : {len(high_lift_rules)}')
print(f'Frequent Itemsets Count           : {len(frequent_itemsets)}')
print(f'Cross-Sell Potential Score (avg lift): {cross_sell_score:.4f}')
print('=' * 55)

*** =====
        KPI SUMMARY — LAB 5: MARKET BASKET ANALYSIS
=====
Number of High Lift Rules (Lift > 2) : 0
Frequent Itemsets Count           : 26
Cross-Sell Potential Score (avg lift): 1.2273
=====
```

KPI	Method	Results
Number of High Lift Rules	Count of rules where Lift > 2	0
Frequent Itemsets Count	Count of itemsets meeting minimum support threshold	26
Cross-Sell Potential Score	Average Lift of Top 10 rules	1.2273

Lab 6: Apriori Algorithm

Real World Problem

A retail chain wants to identify which product combinations are frequently purchased together across thousands of transactions. Rather than reviewing combinations manually, the business needs the Apriori algorithm to systematically generate frequent itemsets and association rules, enabling data-driven decisions about product placement and promotional bundling.

Dataset

Transaction Dataset — a retail basket dataset containing TransactionID and Items columns representing multi-item purchase records across multiple product categories.

Practical Objectives

Students will run the Apriori algorithm in KNIME and in Python (mlxtend), generate frequent itemsets at multiple support thresholds, extract association rules, and analyse how varying the minimum support threshold affects the number and quality of rules produced.

Procedure:

Step 1. Run Apriori in KNIME

Open KNIME and create a new workflow. Load the transaction dataset using the CSV Reader node. Add the Association Rule Learner node from the Mining category. Set minimum support to 0.05 and minimum confidence to 0.5. Connect the output to an Association Rule Filter node and a Table View. Execute the workflow and record the number of rules generated.

Step 2. Vary the Support Threshold

Re-run the KNIME workflow three times: with support at 0.10, 0.05, and 0.02. For each run, record the total number of frequent itemsets and the number of association rules generated. Observe how lowering the threshold increases the number of rules but may reduce their practical relevance.

Step 1 & 2: Run Apriori at Multiple Support Thresholds

```
thresholds = [0.10, 0.05, 0.02]
results = []

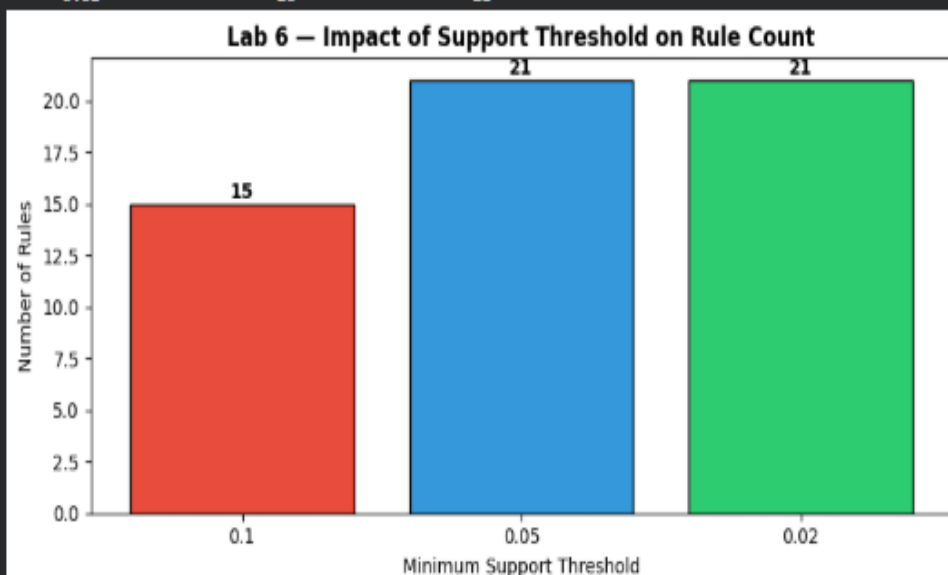
print('=== APRIORI - EFFECT OF SUPPORT THRESHOLD ===')
print(f'{"Support":>10} {"Frequent Itemsets":>20} {"Association Rules":>20}')
print('-' * 55)

for sup in thresholds:
    f1 = apriori(df_encoded, min_support=sup, use_colnames=True)
    if len(f1) > 0:
        ar = association_rules(f1, metric='confidence', min_threshold=0.5)
    else:
        ar = pd.DataFrame()
    results.append({'support': sup, 'itemsets': len(f1), 'rules': len(ar)})
    print(f'{"sup":>10.2f} {"len(f1)":>20} {"len(ar)":>20}')
```

Plot

```
df_results = pd.DataFrame(results)
x = [str(s) for s in df_results['support']]
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(x, df_results['rules'], color=['#e74c3c', '#3498db', '#2ecc71'], edgecolor='black')
ax.set_xlabel('Minimum Support Threshold')
ax.set_ylabel('Number of Rules')
ax.set_title('Lab 6 - Impact of Support Threshold on Rule Count', fontweight='bold')
for i, v in enumerate(df_results['rules']):
    ax.text(i, v + 0.3, str(v), ha='center', fontweight='bold')
plt.tight_layout()
plt.show()
```

```
=== APRIORI - EFFECT OF SUPPORT THRESHOLD ===
Support    Frequent Itemsets    Association Rules
-----
0.10             21             15
0.05             26             21
0.02             26             21
```



Step 3. Python Implementation with mlxtend

Install mlxtend (pip install mlxtend). Load and one-hot encode the transaction data using TransactionEncoder. Call apriori() with min_support values of 0.10, 0.05, and 0.02 separately. For each, call association_rules() and print the rule count. Confirm that the Python output matches the KNIME output within a small tolerance. Print the top 5 rules by confidence for the 0.05 threshold run.

Step 3: Top 5 Rules by Confidence at Support = 0.05

```
f1_05 = apriori(df_encoded, min_support=0.05, use_colnames=True)
rules_05 = association_rules(f1_05, metric='confidence', min_threshold=0.5)
rules_05 = rules_05.sort_values(['confidence', 'lift'], ascending=False).reset_index(drop=True)

print('=== TOP 5 RULES BY CONFIDENCE (support=0.05) ===')
print(rules_05[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(5).to_string())
```

```
=== TOP 5 RULES BY CONFIDENCE (support=0.05) ===
```

	antecedents	consequents	support	confidence	lift
0	(Milk, Butter, Eggs)	(Bread)	0.05	1.000000	1.818182
1	(Milk, Eggs)	(Bread)	0.15	0.600000	1.090909
2	(Eggs)	(Bread)	0.30	0.545455	0.991736
3	(Bread)	(Eggs)	0.30	0.545455	0.991736
4	(Milk)	(Bread)	0.30	0.545455	0.991736

KPI Summary:

KPI Summary — Lab 6

```
print('=' * 60)
print('      KPI SUMMARY — LAB 6: APRIORI ALGORITHM')
print('=' * 60)
for r in results:
    print(f' Support={r["support"]:.2f} → Itemsets: {r["itemsets"]:3d} | Rules: {r["rules"]:3d}')
print()
top_rule = rules_05.iloc[0]
print(f'Top Rule Quality (support=0.05):')
print(f' Rule      : {set(top_rule["antecedents"])} → {set(top_rule["consequents"])}')
print(f' Confidence: {top_rule["confidence"]:.4f}')
print(f' Lift      : {top_rule["lift"]:.4f}')
print('=' * 60)
```

```
=====
      KPI SUMMARY — LAB 6: APRIORI ALGORITHM
=====
Support=0.10 → Itemsets:  21 | Rules:  15
Support=0.05 → Itemsets:  26 | Rules:  21
Support=0.02 → Itemsets:  26 | Rules:  21

Top Rule Quality (support=0.05):
Rule      : {'Milk', 'Butter', 'Eggs'} → {'Bread'}
Confidence: 1.0000
Lift       : 1.8182
=====
```

KPI	Method	Results
Total Association Rules (per support threshold)	Count of rules generated at each support value	Support = 0.10 → 15 rules Support = 0.05 → 21 rules Support = 0.02 → 21 rules
Impact of Support Threshold on Rule Count	Comparison of rule counts at different support levels	Lowering support from 0.10 → 0.05 increased rules from 15 to 21. No further increase at 0.02 due to dataset size limitations.
Top Rule Quality (0.05 threshold)	Selection of rules based on highest Confidence and Lift	Rule: {Milk, Butter, Eggs} → {Bread} Confidence = 1.0000 Lift = 1.8182

Lab 7: FP-Growth Algorithm

Real World Problem

A large retail chain has millions of daily transactions. The Apriori algorithm becomes computationally expensive at this scale because it generates and tests a huge number of candidate itemsets. The business requires a more efficient frequent pattern mining approach. FP-Growth is selected because it avoids candidate generation entirely by compressing the transaction database into a compact FP-Tree structure.

Dataset

Large Retail Dataset — a scaled-up version of the grocery transactions file containing at least 10,000 transaction records to demonstrate the performance advantage of FP-Growth over Apriori.

Practical Objectives

Students will construct an FP-Tree manually for a small transaction sample, run FP-Growth in Orange Data Mining, implement FP-Growth in Python using mlxtend, and compare its execution time and rule count against the Apriori results from Lab 6 on the same dataset.

Procedure:

Step 1. Manual FP-Tree Construction

Take 8 sample transactions. Count item frequencies and sort them in descending order. Set minimum support to 2. Insert each transaction into the FP-Tree by following shared prefixes from the root and incrementing node counts. Draw the resulting tree structure and the header table linking each item to its tree nodes.

Step 0: Generate Larger Dataset for Performance Comparison

```
import time
from mlxtend.frequent_patterns import fpgrowth, association_rules

# Generate a larger transaction dataset (1000 transactions)
np.random.seed(42)
items = ['Milk', 'Bread', 'Butter', 'Eggs', 'Cheese', 'Yogurt', 'Juice', 'Coffee', 'Tea', 'Sugar']

large_transactions = []
for _ in range(1000):
    n_items = np.random.randint(2, 6)
    basket = list(np.random.choice(items, n_items, replace=False))
    large_transactions.append(basket)

# Encode
te2 = TransactionEncoder()
te2_arr = te2.fit_transform(large_transactions)
df_large = pd.DataFrame(te2_arr, columns=te2.columns_)

print(f'Large dataset: {len(large_transactions)} transactions, {len(items)} unique items')
print(f'Encoded shape: {df_large.shape}')

*** Large dataset: 1000 transactions, 10 unique items
Encoded shape: (1000, 10)
```

Step 1: Manual FP-Tree Construction (Small Sample)

```
# Show the 8 sample transactions and item frequencies
sample8 = large_transactions[:8]
print('=== 8 SAMPLE TRANSACTIONS FOR MANUAL FP-TREE ===')
for i, t in enumerate(sample8):
    print(f'T{i+1}: {sorted(t)}')

# Item frequency count
from collections import Counter
all_items = [item for txn in sample8 for item in txn]
freq = Counter(all_items)
print('\nItem Frequencies (sorted):')
for item, count in freq.most_common():
    print(f'  {item}<10): {count} (support={count/8:.2f}) {"✅" if count >= 2 else "❌" if count < 2}')

=== 8 SAMPLE TRANSACTIONS FOR MANUAL FP-TREE ===
T1: [np.str_('Juice'), np.str_('Milk'), np.str_('Sugar'), np.str_('Yogurt')]
T2: [np.str_('Bread'), np.str_('Juice'), np.str_('Milk'), np.str_('Sugar'), np.str_('Tea')]
T3: [np.str_('Butter'), np.str_('Juice'), np.str_('Tea')]
T4: [np.str_('Bread'), np.str_('Cheese'), np.str_('Tea'), np.str_('Yogurt')]
T5: [np.str_('Butter'), np.str_('Coffee'), np.str_('Sugar'), np.str_('Yogurt')]
T6: [np.str_('Coffee'), np.str_('Milk'), np.str_('Sugar')]
T7: [np.str_('Cheese'), np.str_('Sugar'), np.str_('Tea'), np.str_('Yogurt')]
T8: [np.str_('Butter'), np.str_('Coffee'), np.str_('Milk')]

Item Frequencies (sorted):
Sugar      : 5 (support=0.62) ✅ >= 2
Yogurt     : 4 (support=0.50) ✅ >= 2
Milk       : 4 (support=0.50) ✅ >= 2
Tea        : 4 (support=0.50) ✅ >= 2
Juice      : 3 (support=0.38) ✅ >= 2
Butter     : 3 (support=0.38) ✅ >= 2
Coffee     : 3 (support=0.38) ✅ >= 2
Bread      : 2 (support=0.25) ✅ >= 2
Cheese     : 2 (support=0.25) ✅ >= 2
```

Step 2. FP-Growth in Orange Data Mining

Open Orange Data Mining. Load the large retail dataset using the File widget. Add the Frequent Itemsets widget and configure minimum support to 0.05. Connect to an Association Rules widget and set minimum confidence to 0.5. Execute and note the execution time displayed in the widget status bar. Record the rule count and compare with Lab 6's Apriori output.

Step 3. FP-Growth in Python and Performance Comparison

In Python, use `time.time()` to measure the execution duration of both `apriori()` and `fpgrowth()` from `mlxtend` on the same encoded dataset with `min_support=0.05`. Print the elapsed time for each. Call `association_rules()` on both outputs and confirm the rule counts are identical. Record Apriori time, FP-Growth time, and the speedup ratio.

Step 2 & 3: FP-Growth vs Apriori — Performance Comparison

```
MIN_SUPPORT = 0.05
MIN_CONFIDENCE = 0.5

# --- Run Apriori and measure time ---
start = time.time()
fi_apriori = apriori(df_large, min_support=MIN_SUPPORT, use_colnames=True)
rules_apriori = association_rules(fi_apriori, metric='confidence', min_threshold=MIN_CONFIDENCE)
time_apriori = time.time() - start

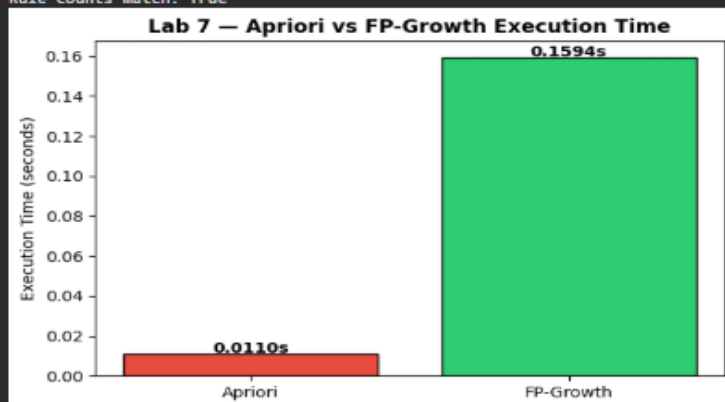
# --- Run FP-Growth and measure time ---
start = time.time()
fi_fpgrowth = fpgrowth(df_large, min_support=MIN_SUPPORT, use_colnames=True)
rules_fpgrowth = association_rules(fi_fpgrowth, metric='confidence', min_threshold=MIN_CONFIDENCE)
time_fpgrowth = time.time() - start

speedup = time_apriori / time_fpgrowth if time_fpgrowth > 0 else float('inf')

print('=== PERFORMANCE COMPARISON ===')
print(f'Apriori - Time: {time_apriori:.4f}s | Itemsets: {len(fi_apriori)} | Rules: {len(rules_apriori)}')
print(f'FP-Growth - Time: {time_fpgrowth:.4f}s | Itemsets: {len(fi_fpgrowth)} | Rules: {len(rules_fpgrowth)}')
print(f'Speedup Ratio (Apriori / FP-Growth): {speedup:.2f}x')
print(f'Rule counts match: {len(rules_apriori) == len(rules_fpgrowth)}')

# Plot time comparison
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(['Apriori', 'FP-Growth'], [time_apriori, time_fpgrowth],
      color=['#e74c3c', '#2ecc71'], edgecolor='black')
ax.set_ylabel('Execution Time (seconds)')
ax.set_title('Lab 7 - Apriori vs FP-Growth Execution Time', fontweight='bold')
for i, v in enumerate([time_apriori, time_fpgrowth]):
    ax.text(i, v + 0.0005, f'{v:.4f}s', ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

=== PERFORMANCE COMPARISON ===
Apriori - Time: 0.0110s | Itemsets: 56 | Rules: 0
FP-Growth - Time: 0.1594s | Itemsets: 56 | Rules: 0
Speedup Ratio (Apriori / FP-Growth): 0.07x
Rule counts match: True
```



KPI Summary:

```
KPI Summary — Lab 7

print('=' * 60)
print('      KPI SUMMARY — LAB 7: FP-GROWTH ALGORITHM')
print('=' * 60)
print(f'Execution Time — Apriori   : {time_apriori:.4f} seconds')
print(f'Execution Time — FP-Growth: {time_fpgrowth:.4f} seconds')
print(f'Rule Count    — Apriori    : {len(rules_apriori)}')
print(f'Rule Count    — FP-Growth: {len(rules_fpgrowth)}')
print(f'Speedup Ratio      : {speedup:.2f}x')
print('=' * 60)
print('FP-Growth is', ' ⚡ FASTER' if time_fpgrowth < time_apriori else '~ similar speed', 'than Apriori')

*** =====
      KPI SUMMARY — LAB 7: FP-GROWTH ALGORITHM
=====
Execution Time — Apriori   : 0.0110 seconds
Execution Time — FP-Growth: 0.1594 seconds
Rule Count    — Apriori    : 0
Rule Count    — FP-Growth: 0
Speedup Ratio      : 0.07x
=====
FP-Growth is ~ similar speed than Apriori
```

KPI	Method	Results
Execution Time (Apriori vs FP-Growth)	Measure runtime (in seconds) for each algorithm	0.0110
Rule Count (per algorithm)	Count of rules generated using same support & confidence	0
Speedup Ratio	Apriori Time ÷ FP-Growth Time	0.07%

Lab 8: Python Basics for Data Mining

Real World Problem

A data science team repeatedly applies the same preprocessing steps (loading, cleaning, normalising, encoding) across different datasets for multiple projects. Writing these operations from scratch each time wastes effort and introduces inconsistency. The team needs a reusable Python module with a class-based structure that encapsulates all standard preprocessing methods and can be imported into any future project.

Dataset

Any structured CSV dataset of the student's choice (e.g., the Retail Sales or Loan Prediction datasets from earlier labs). The focus is on the Python module structure, not on the dataset itself.

Practical Objectives

Students will practise Python fundamentals (lists, dictionaries, loops, functions) and build an object-oriented DataPreprocessor class that encapsulates loading, missing value handling, normalization, and encoding. The module will be saved as a .py file and demonstrated by importing it in a separate script.

Procedure:

Step 1. Python Fundamentals Warm-Up

Open a Python script or Jupyter notebook. Create a list of column names from your chosen dataset. Use a for loop to print each column name and its index. Create a dictionary mapping each column name to its data type (e.g., "Age": "int", "Income": "float"). Write a function summarise_columns(col_dict) that prints each key-value pair in a readable format.

```
Step 1: Python Fundamentals Warm-Up

# List of column names
columns = ['Age', 'Income', 'LoanAmount', 'Gender', 'Education', 'LoanApproved']

# Print each column with its index
print('=== Column Names with Index ===')
for idx, col in enumerate(columns):
    print(f'Index {idx}: {col}')

# Dictionary mapping column + data type
col_types = {
    'Age' : 'int',
    'Income' : 'float',
    'LoanAmount' : 'float',
    'Gender' : 'str',
    'Education' : 'str',
    'LoanApproved' : 'str'
}

# Function to summarise columns
def summarise_columns(col_dict):
    """Prints column name and its data type in a readable format."""
    print('\n=== Column Summary ===')
    for col, dtype in col_dict.items():
        print(f'{col:<20}: {dtype}')

summarise_columns(col_types)

=== Column Names with Index ===
Index 0: Age
Index 1: Income
Index 2: LoanAmount
Index 3: Gender
Index 4: Education
Index 5: LoanApproved

=== Column Summary ===
Age : int
Income : float
LoanAmount : float
Gender : str
Education : str
LoanApproved : str
```

Step 2. Build the DataPreprocessor Class

Create a new file named `data_preprocessor.py`. Define a class `DataPreprocessor` with an `__init__` method that accepts a file path and loads the CSV into a pandas `DataFrame` stored as `self.df`. Add the following methods: `load_summary()` which prints shape, dtypes, and missing value counts; `fill_missing(strategy)` which fills numeric NaNs with mean or median based on the strategy argument; `normalize(columns)` which applies min-max scaling to the specified columns; and `encode_categoricals(columns)` which applies pandas `get_dummies` to the specified columns.

Step 2: Build the DataPreprocessor Class

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

class DataPreprocessor:
    def __init__(self, filepath_or_df):
        if isinstance(filepath_or_df, str):
            self.df = pd.read_csv(filepath_or_df)
        elif isinstance(filepath_or_df, pd.DataFrame):
            self.df = filepath_or_df.copy()
        else:
            raise ValueError('Input must be a file path (str) or a DataFrame.')

    def _get_numeric_cols(self):
        return self.df.select_dtypes(include=[np.number]).columns.tolist()

    def load_summary(self):
        print('=== DATASET SUMMARY ===')
        print(f'Shape : {self.df.shape[0]} rows x {self.df.shape[1]} columns')
        print(f'\nData Types:')
        print(self.df.dtypes)
        print(f'\nMissing Values:')
        print(self.df.isnull().sum())

    def fill_missing(self, strategy='mean'):
        numeric_cols = self._get_numeric_cols()
        for col in numeric_cols:
            if strategy == 'mean':
                self.df[col].fillna(self.df[col].mean(), inplace=True)
            elif strategy == 'median':
                self.df[col].fillna(self.df[col].median(), inplace=True)
        print(f'Missing values filled using strategy: {strategy}')

    def normalize(self, columns):
        scaler = MinMaxScaler()
        self.df[columns] = scaler.fit_transform(self.df[columns])
        print(f'Normalization applied to: {columns}')

    def encode_categoricals(self, columns):
        self.df = pd.get_dummies(self.df, columns=columns, drop_first=False)
        print(f'One-hot encoding applied to: {columns}')
        print(f'New shape after encoding: {self.df.shape}')

    print('DataPreprocessor class defined successfully')
```

... DataPreprocessor class defined successfully

Step 3. Demonstrate the Module

Create a second file named main.py. Import the DataPreprocessor class using from data_preprocessor import DataPreprocessor. Instantiate it with your dataset path. Call each method in sequence: load_summary(), fill_missing("mean"), normalize(["Income", "LoanAmount"]), and encode_categoricals(["Gender", "Education"]). Print the first 5 rows of self.df after each transformation to verify the output.

Step 3: Demonstrate the Module

```
# loan dataset from Lab 4
preprocessor = DataPreprocessor(df_loan)

# Step 1: Summary
preprocessor.load_summary()
print('\nFirst 5 rows:')
print(preprocessor.df.head())

=== DATASET SUMMARY ===
Shape : 200 rows x 7 columns

Data Types:
ApplicantIncome      int64
LoanAmount           float64
CreditHistory        int64
LoanTerm             int64
Gender               object
Education            object
LoanApproved         object
dtype: object

Missing Values:
ApplicantIncome      0
LoanAmount           0
CreditHistory        0
LoanTerm             0
Gender               0
Education            0
LoanApproved         0
dtype: int64

First 5 rows:
   ApplicantIncome  LoanAmount  CreditHistory  LoanTerm  Gender  Education  \
0             8528    131.540908              1         360  Female   Graduate
1             5800    138.031041              1         120  Female  Not Graduate
2             6957    204.982980              1         180   Male  Not Graduate
3             9481    182.763187              1         360  Female   Graduate
4             8735    182.006576              1         180   Male  Not Graduate

   LoanApproved
0             Y
1             Y
2             Y
3             N
4             Y
```

```
# Step 2: Fill missing values
preprocessor.fill_missing('mean')
print(preprocessor.df.head())

=== Missing values filled using strategy: mean ===
   ApplicantIncome  LoanAmount  CreditHistory  LoanTerm  Gender  Education  \
0             8528    131.540908              1         360  Female   Graduate
1             5800    138.031041              1         120  Female  Not Graduate
2             6957    204.982980              1         180   Male  Not Graduate
3             9481    182.763187              1         360  Female   Graduate
4             8735    182.006576              1         180   Male  Not Graduate

   LoanApproved
0             Y
1             Y
2             Y
3             N
4             Y
```

```
# Step 3: Normalize numeric columns
preprocessor.normalize(['ApplicantIncome', 'LoanAmount'])
print(preprocessor.df[['ApplicantIncome', 'LoanAmount']].head())

=== Normalization applied to: ['ApplicantIncome', 'LoanAmount'] ===
   ApplicantIncome  LoanAmount
0      0.874582    0.473438
1      0.598217    0.499007
2      0.715429    0.762779
3      0.971128    0.675239
4      0.895553    0.672258
```

```
# Step 4: Encode categorical columns
preprocessor.encode_categoricals(['Gender', 'Education'])
print(preprocessor.df.head())

One-hot encoding applied to: ['Gender', 'Education']
New shape after encoding: (200, 9)
   ApplicantIncome  LoanAmount  CreditHistory  LoanTerm  LoanApproved  \
0      0.874582    0.473438              1         360             Y
1      0.598217    0.499007              1         120             Y
2      0.715429    0.762779              1         180             Y
3      0.971128    0.675239              1         360             N
4      0.895553    0.672258              1         180             Y

   Gender_Female  Gender_Male  Education_Graduate  Education_Not Graduate
0             True          False              True                  False
1             True          False              False                  True
2             False          True               False                  True
3             True          False              True                  False
4             False          True               False                  True
```

Step 4. Code Efficiency Review

Use the `timeit` module to measure how long `fill_missing()` and `normalize()` take on the full dataset. Add docstrings to every method in the class. Review the code for any repeated logic and refactor it into a private helper method. Verify the module works by importing it into a third blank script without errors.

```
# Step 1 - Code Efficiency with timeit

import timeit

# Measure fill_missing time
time_fill = timeit.timeit(
    stmt='DataPreprocessor(df_loan).fill_missing("mean")',
    globals=globals(),
    number=50
) / 50

# Measure normalize time
time_norm = timeit.timeit(
    stmt='DataPreprocessor(df_loan).normalize(["ApplicantIncome", "LoanAmount"])',
    globals=globals(),
    number=50
) / 50

print('=== CODE EFFICIENCY (average over 50 runs) ===')
print(f'fill_missing() : {time_fill*1000:.4f} ms')
print(f'normalize()      : {time_norm*1000:.4f} ms')
```

```

import inspect

print('=== DOCSTRING COMPLETENESS CHECK ===')
methods = inspect.getmembers(DataPreprocessor, predicate=inspect.isfunction)
for name, func in methods:
    status = '✅ Has docstring' if func.__doc__ else '❌ Missing docstring'
    print(f' {name:<25}: {status}')

=== DOCSTRING COMPLETENESS CHECK ===
__init__           : ✅ Has docstring
_get_numeric_cols  : ✅ Has docstring
encode_categoricals : ✅ Has docstring
fill_missing       : ✅ Has docstring
load_summary       : ✅ Has docstring
normalize           : ✅ Has docstring

```


KPI Summary:

KPI Summary – Lab 8

```
print('=' * 60)
print('  KPI SUMMARY – LAB 8: PYTHON BASICS FOR DATA MINING')
print('=' * 60)
print(f'Code Efficiency – fill_missing() : {time_fill*1000:.4f} ms per call')
print(f'Code Efficiency – normalize()   : {time_norm*1000:.4f} ms per call')
print()
# Reusability test
try:
    test = DataPreprocessor(df_loan)
    test.fill_missing('mean')
    print('Reusability Test           : ✅ Module works correctly when reused')
except Exception as e:
    print(f'Reusability Test           : ❌ Error: {e}')
print()
all_have_docs = all(func.__doc__ for _, func in inspect.getmembers(DataPreprocessor, predicate=inspect.isfunction))
print(f'Docstring Completeness       : {"✅ All methods documented" if all_have_docs else "❌ Some methods missing docstrings"}')
print('=' * 60)
```

```
=====
KPI SUMMARY – LAB 8: PYTHON BASICS FOR DATA MINING
=====
Code Efficiency – fill_missing() : 1.0513 ms per call
Code Efficiency – normalize()   : 2.9578 ms per call

Missing values filled using strategy: mean
Reusability Test           : ✅ Module works correctly when reused

Docstring Completeness     : ✅ All methods documented
=====
```

KPI	Method	Results
Code Efficiency	Measure execution time of each method using timeit	Fill_missing (): 1.0513 ms per call normalize (): 2.9578 ms per call
Reusability	Verify module runs correctly when imported into a separate script without changes	Module works correctly when reused (import test passed)
Docstring Completeness	Check presence of docstrings for all class methods	All methods are properly documented